

A Hybrid Data Driven and Distributionally Robust Optimization Framework for Optimal Sizing of PV-BESS Microgrids in Uncertain and Data-Scarce Environment

A Project Report

Submitted in Fulfillment of the Requirements for the Degree of

Bachelor of Technology in Computer Science and Engineering (Data Science)

Submitted by

Naman Chaudhary (2200271540071)

Navendu Shekhar (2200271540073)

Nikhil Kumar (2200271540074)

Ritvik Sharma (2200271540088)

Under the Supervision of

Dr. Ashish Dixit



**Department of Computer Science & Engineering
AJAY KUMAR GARG ENGINEERING COLLEGE, GHAZIABAD
DR. A.P.J. ABDUL KALAM TECHNICAL UNIVERSITY,
LUCKNOW**

May, 2026

Declaration

We hereby declare that this project report entitled “**A Hybrid Data Driven and Distributionally Robust Optimization Framework for Optimal Sizing of PV-BESS Microgrids in Uncertain and Data-Scarce Environment**” submitted to the Department of Computer Science and Engineering, Ajay Kumar Garg Engineering College, Ghaziabad in partial fulfillment of the requirements for the degree of *Bachelor of Technology* is an authentic record of our original work carried out during the academic year 2022–2026.

The work contained in this report has not been submitted in part or in full to any other university or institution for the award of any degree or diploma. All sources of information used in this report have been duly acknowledged and referenced.

We have self-checked this report for plagiarism using appropriate tools and confirm that it meets the prescribed academic integrity standards.

Student Names

Signatures

Naman Chaudhary (2200271540071) _____

Navendu Shekhar (2200271540073) _____

Nikhil Kumar (2200271540074) _____

Ritvik Sharma (2200271540088) _____

Date: May, 2026

Certificate

This is to certify that the project report entitled “**A Hybrid Data Driven and Distributionally Robust Optimization Framework for Optimal Sizing of PV-BESS Microgrids in Uncertain and Data-Scarce Environment**”, submitted by the following students in partial fulfillment of the requirements for the award of the degree of *Bachelor of Technology in Computer Science and Engineering* to Ajay Kumar Garg Engineering College, Ghaziabad (affiliated to Dr. A.P.J. Abdul Kalam Technical University, Lucknow) is an authentic record of genuine work carried out by them under my supervision.

Students:

Naman Chaudhary (2200271540071)

Navendu Shekhar (2200271540073)

Nikhil Kumar (2200271540074)

Ritvik Sharma (2200271540088)

The matter presented in this report has not been submitted for the award of any other degree or diploma at this or any other institution.

Dr. Ashish Dixit

Associate Professor

Department of Computer Science & Engineering

AJAY KUMAR GARG ENGINEERING
COLLEGE, GHAZIABAD

Dr. Anu Chaudhary

Head of Department

Department of Computer Science & Engineering

AJAY KUMAR GARG ENGINEERING
COLLEGE, GHAZIABAD

Date: May, 2026

Acknowledgements

We express our deepest gratitude to our project supervisor, **Dr. Ashish Dixit**, Department of Computer Science & Engineering, Ajay Kumar Garg Engineering College, Ghaziabad, for their invaluable guidance, constant encouragement, and constructive feedback throughout the course of this project. Their expertise and insights have been instrumental in shaping this work.

We would like to extend our sincere thanks to **Dr. Amita Dev**, Advisor, for her valuable suggestions, guidance, and support, which greatly contributed to the successful completion of this project.

We are highly grateful to **Dr. Hemant Ahuja**, Director, Ajay Kumar Garg Engineering College, Ghaziabad, for providing institutional support and fostering an environment that encourages innovation and research.

We sincerely thank **Dr. Anu Chaudhary**, Head of the Department of Computer Science & Engineering, for providing the necessary infrastructure, lab facilities, and an environment conducive to research and learning.

We extend our heartfelt thanks to the faculty members of the Department of Computer Science & Engineering for their continuous support and for imparting the foundational knowledge that has made this project possible.

We are grateful to the management of **AJAY KUMAR GARG ENGINEERING COLLEGE, GHAZIABAD** for maintaining an ecosystem that encourages innovation and applied research.

Finally, we acknowledge our families and friends for their unwavering moral support and patience during the entire duration of this project.

Student Names:

Naman Chaudhary (2200271540071)

Navendu Shekhar (2200271540073)

Nikhil Kumar (2200271540074)

Ritvik Sharma (2200271540088)

Date: May, 2026

Abstract

Photovoltaic (PV) systems and Battery Energy Storage Systems (BESS) must be optimally sized to ensure flexible power delivery at minimum cost, especially under conditions of high uncertainty. Rapid growth in renewable energy adoption has exposed critical gaps in conventional planning tools, particularly in regions where historical energy data is sparse or unreliable. This study introduces a hybrid framework that integrates data-driven Machine Learning (ML) with Distributionally Robust Optimization (DRO) to address the challenges of optimal PV-BESS sizing in uncertain and data-scarce environments.

Traditional deterministic methods often fail to capture the volatility of renewable generation and load demand, while standard stochastic models require extensive historical datasets that are rarely available in rural or remote regions. The proposed methodology utilizes ML techniques, including regression-based forecasting and clustering algorithms, to predict demand and supply patterns, which directly inform the construction of ambiguity sets for the DRO model. This approach allows for a “distributionally strong” optimization that protects against worst-case scenarios and distributional shifts without requiring a fixed probability distribution.

The framework is validated through the column-and-constraint generation (CnCG) algorithm, which efficiently solves the resulting robust optimization problem. Experimental results demonstrate that the hybrid framework achieves a 15–20% reduction in total lifecycle costs compared to traditional deterministic approaches. Furthermore, the system shows a 10–15% increase in the robustness margin, significantly reducing the likelihood of black-outs. By reducing energy loss by up to 50% and improving overall system reliability to 92%, this framework provides a cost-effective and resilient solution for sustainable electrification in data-scarce and off-grid regions.

Keywords: Photovoltaic (PV) Systems, Battery Energy Storage System (BESS), Optimal Sizing, Distributionally Robust Optimization (DRO), Data-Driven Optimization, Renewable Energy Integration, Machine Learning, Data-Scarce Environments.

Contents

Declaration	i
Certificate	ii
Acknowledgements	iii
Abstract	iv
List of Figures	x
List of Tables	xi
1 Introduction	1
1.1 Background and Motivation	1
1.1.1 The Global Electrification Gap	1
1.1.2 The Cost–Reliability Trade-off	1
1.1.3 Why Hybrid ML–DRO?	2
1.2 Problem Statement	2
1.3 Research Objectives	3
1.4 Novelty and Originality	4
1.5 Scope of the Project	5
1.6 Contributions of This Thesis	6
1.7 Report Organization	7
2 Literature Review	8
2.1 Historical Overview of Microgrid Optimization	8
2.2 Machine Learning in Energy-Systems Forecasting	8
2.2.1 Solar Irradiance Forecasting	8
2.2.2 Load-Demand Forecasting	9
2.3 Optimization Methods for Microgrid Sizing	9
2.3.1 Deterministic Optimization	9
2.3.2 Stochastic Programming	9
2.3.3 Robust Optimization	10

2.3.4	Distributionally Robust Optimization	10
2.4	Comparative Analysis of Existing Approaches	10
2.5	Identified Research Gap	11
3	Research Methodology	12
3.1	Research Approach	12
3.2	Proposed Framework	12
3.3	Research Design	13
3.4	Data Collection and Dataset Preparation	14
3.5	Data Preprocessing	15
3.6	Random Forest Machine Learning Model	16
3.7	Distributionally Robust Optimization Model	17
3.8	PV-BESS Sizing Formulation	17
3.9	Objective Function	18
3.10	Reliability Constraints	18
3.11	Solution Methodology	19
3.12	Comparative Benchmark Models	20
3.13	Sensitivity Analysis	20
4	Implementation	21
4.1	Introduction	21
4.2	Overall System Architecture	22
4.2.1	Application Entry Point: <code>main.py</code>	23
4.3	Random Forest Implementation	25
4.3.1	Random Forest Training Flow	25
4.3.2	Core ML Model Module: <code>core/ml_model.py</code>	26
4.4	DRO Optimisation Engine	30
4.4.1	DRO Optimisation Workflow	30
4.4.2	DRO Engine Module: <code>core/dro_engine.py</code>	31
4.5	Solar Irradiance Data Module	35
4.5.1	Solar Data Module: <code>core/solar_data.py</code>	36
4.6	Backend System Architecture	38
4.6.1	DRO Optimisation Endpoint: <code>api/optimize.py</code>	39
4.6.2	ML Prediction Endpoint: <code>api/predict.py</code>	41
4.6.3	Combined Simulation Endpoint: <code>api/simulate.py</code>	42
4.6.4	Solar Data Endpoint: <code>api/solar.py</code>	46
4.6.5	Health Check Endpoint: <code>api/health.py</code>	48
4.7	Frontend Design	49
4.8	Deployment Workflow	52

4.9	System Outputs	52
4.10	Implementation Validation	53
5	Results and Discussion	54
5.1	Experimental Setup	54
5.1.1	Hardware and Software Environment	55
5.1.2	Dataset Description	55
5.2	Machine Learning Forecast Accuracy	55
5.2.1	Random Forest Result Table	56
5.2.2	Discussion of Machine Learning Results	57
5.3	Feature Importance Analysis	57
5.4	DRO Optimization Results	58
5.5	Benchmark Comparison	59
5.6	Cost-Reliability Trade-off Analysis	60
5.7	Sensitivity Analysis	60
5.8	Sensitivity to Data Availability	61
5.9	Computational Performance	61
5.10	Discussion of Findings	62
5.11	Limitations of Results	62
6	Societal Impact and SDG Alignment	63
6.1	Introduction	63
6.2	Societal Relevance of PV-BESS Microgrids	63
6.3	Contribution to Rural Electrification	63
6.4	Economic Impact	64
6.5	Environmental Impact	65
6.6	Social Impact	65
6.7	Primary SDG Alignment	66
6.7.1	SDG 7: Affordable and Clean Energy	66
6.7.2	SDG 9: Industry, Innovation and Infrastructure	66
6.7.3	SDG 13: Climate Action	66
6.8	Secondary SDG Linkages	66
6.8.1	SDG 3: Good Health and Well-Being	66
6.8.2	SDG 4: Quality Education	67
6.8.3	SDG 8: Decent Work and Economic Growth	67
6.8.4	SDG 11: Sustainable Cities and Communities	67
6.8.5	SDG 12: Responsible Consumption and Production	67
6.9	Stakeholder Impact Analysis	67
6.9.1	Rural Communities	67

6.9.2	Microgrid Engineers	67
6.9.3	Policymakers	67
6.9.4	Environmental Agencies	68
6.9.5	Researchers	68
6.10	Ethical Considerations	68
6.11	Limitations in Societal Deployment	68
7	Universal Human Values and Harmony with Nature	69
7.1	Introduction	69
7.2	Relevance of Universal Human Values	69
7.3	Harmony with Nature	70
7.4	Material Order: Efficient Use of Physical Resources	70
7.5	Plant Order: Protecting Natural Ecosystems	70
7.6	Animal Order: Minimizing Ecological Disturbance	71
7.7	Human Order: Advancing Human Dignity and Capability	71
7.8	Responsible Technology Development	71
7.9	Mutual Fulfillment: Technology in Service of Life	72
7.10	Ethical Reflection on the Proposed Work	72
8	Conclusion and Future Scope	73
8.1	Summary of the Work	73
8.2	Major Conclusions	73
8.3	Technical Contributions	74
8.4	Research Contributions	75
8.5	Practical Contributions	75
8.6	Key Findings	76
8.7	Limitations of the Study	76
8.8	Future Scope	77
8.8.1	Use of Real-World Datasets	77
8.8.2	Integration of Additional Renewable Sources	77
8.8.3	Advanced Machine Learning Models	77
8.8.4	Multi-Objective Optimization	77
8.8.5	Grid-Connected Microgrid Extension	77
8.8.6	Real-Time Decision Support	77
8.8.7	Cloud Deployment	77
8.8.8	Battery Degradation Modelling	78
8.9	Closing Remarks	78
	References	79

A	Research Contribution	81
A.1	Patent Application Snapshot	81
B	SDG Compliance Certificate	82
C	Project Snapshots and Description	83
C.1	Module 1: Data Preprocessing and Forecasting	83
C.2	Module 2: Uncertainty Modeling	84
C.3	Module 3: Robust Optimization (CnCG Iteration)	85
D	Student Curriculum Vitae	86
E	Poster	95

List of Figures

4.1	Hybrid ML–DRO Framework Architecture for Optimal PV–BESS Microgrid Sizing	22
4.2	Random Forest Training Pipeline	26
4.3	Operational Workflow of Distributionally Robust Optimization for Optimal PV–BESS Sizing under Uncertainty	31
4.4	Backend System Architecture	39
4.5	Location Selection Interface with Solar Irradiance Dataset and Filters	50
4.6	System Configuration Panel for Energy and Optimization Parameters	50
4.7	Optimization Results Dashboard for PV–BESS Sizing and Performance Metrics	51
4.8	Comparative Visualization of Deterministic, Robust, and ML–DRO Methods	51
4.9	Deployment Workflow	52
A.1	Official Patent Filing Receipt (G.A.R.6) for Hybrid ML–DRO PV–BESS Optimization Framework	81

List of Tables

2.1	Comparative Analysis of Microgrid Optimization Methodologies	10
5.1	Experimental Platform Specifications	55
5.2	Random Forest Forecast Accuracy on Test Data	56
5.3	Random Forest Aggregate Feature Importances	57
5.4	Reference Scenario Inputs	58
5.5	Reference Scenario DRO Optimization Output	58
5.6	Comparison of Optimization Methods	59
5.7	Cost-Reliability Trade-off Curve	60
5.8	Sensitivity to DRO Robustness Parameter	60
5.9	Sensitivity to Historical Data Availability	61
5.10	Endpoint Latencies	61

Chapter 1

Introduction

1.1 Background and Motivation

The twenty-first century has witnessed an unprecedented global transition toward sustainable, low-carbon energy systems, driven by mounting concerns over climate change, energy security, and the imperative of universal electricity access. Within this broader transformation, microgrids—localized energy networks capable of operating independently of the central grid or in synchronization with it—have emerged as a strategically important infrastructure paradigm. In particular, the integration of Photovoltaic (PV) generation with Battery Energy Storage Systems (BESS) offers a technically mature and increasingly economically viable pathway for providing reliable electricity to rural, remote, and off-grid communities where extending the centralized transmission grid is either physically infeasible or prohibitively expensive [1, 2].

1.1.1 The Global Electrification Gap

According to the International Energy Agency, approximately 733 million people worldwide still lacked access to electricity as of 2020, with the majority concentrated in sub-Saharan Africa and South Asia [1]. Standalone PV-BESS microgrids represent a compelling solution in such contexts: solar irradiance is abundant in many energy-poor regions, battery technology costs have declined by over 90% in the past decade [3], and the modular nature of PV-BESS systems enables incremental capacity expansion as community needs evolve. Lithium-ion battery pack prices, which exceeded USD 1,100/kWh in 2010, fell below USD 140/kWh by 2023, fundamentally altering the economics of off-grid renewable deployment.

1.1.2 The Cost–Reliability Trade-off

The technically optimal design of such microgrids remains a non-trivial engineering and computational challenge. The fundamental tension in microgrid sizing lies between economic efficiency and operational reliability. An oversized system—one with excess PV capacity and battery storage—minimizes the risk of energy shortages but incurs unnecessary capital expenditure, rendering the project financially unviable. Conversely, an undersized system minimizes upfront costs but exposes the community to frequent power outages, degrading quality of life and undermining trust in renewable-energy solutions. The optimal

design must therefore navigate a cost-reliability trade-off that is inherently sensitive to uncertainty in the underlying energy supply and demand patterns.

This sensitivity to uncertainty is dramatically amplified in data-scarce environments. Robust microgrid design traditionally requires extensive historical records of solar irradiance, ambient temperature, and local load profiles—ideally spanning multiple years to capture seasonal and inter-annual variability. In remote or newly electrified regions, such data simply does not exist, creating what this research terms the “double-blind” problem: the system designer is simultaneously uncertain about both the future realization of energy variables and the statistical distribution governing those variables.

1.1.3 Why Hybrid ML–DRO?

This work is therefore motivated by the pressing practical need to develop optimization frameworks that yield reliable, cost-effective microgrid designs even when historical information is sparse, noisy, or entirely absent. Two methodological streams—machine learning and robust optimization—have each demonstrated partial solutions to this problem in isolation, but their unified application to PV-BESS sizing has received comparatively little attention. Machine learning excels at distilling predictive structure from heterogeneous data sources (satellite imagery, weather reanalyses, ground sensors) but cannot, on its own, deliver the worst-case guarantees demanded by infrastructure planners. Robust optimization, conversely, provides such guarantees but is widely criticized for excessive conservatism and an inability to incorporate predictive information.

The Distributionally Robust Optimization (DRO) paradigm bridges this gap by hedging against an entire *family* of probability distributions consistent with available evidence, rather than against a single estimated distribution or against the absolute worst case. By constructing the ambiguity-set boundary using ML-derived statistical estimates, the resulting framework automatically tightens its conservatism as more data becomes available—a property we term *adaptive conservatism*.

1.2 Problem Statement

The fundamental challenge addressed in this research is the optimal sizing of PV-BESS standalone microgrids under conditions of joint uncertainty in solar generation and load demand, compounded by severe limitations in the availability of historical data.

Formally, let $\mathcal{P}^*$ denote the true but unknown joint probability distribution of solar irradiance ξ_s and load demand ξ_d over a planning horizon $\mathcal{T} = \{1, 2, \dots, T\}$. The optimal sizing problem seeks to determine the PV panel capacity x_{PV} (in kWp) and battery storage capacity x_B (in kWh) such that the expected total system cost over the planning horizon is

minimized, subject to the constraint that the system reliably meets demand under all realizations of (ξ_s, ξ_d) drawn from $\mathcal{P}^*$.

The key complications arise from two sources:

1. **Distributional Uncertainty:** In data-scarce regions, $\mathcal{P}^*$ is unknown. Standard stochastic-programming approaches assume $\mathcal{P}^*$ is known (or can be accurately estimated from data) and optimize expected cost with respect to this assumed distribution. When the assumed distribution is incorrect—which is virtually guaranteed with limited data—the resulting designs may be severely suboptimal or unreliable [4].
2. **Scenario-Generation Scarcity:** Even modern data-driven methods require a minimum volume of training samples to produce reliable statistical estimates. In remote regions, available data may consist of only a few months of measurements, or may be derived from satellite-based proxies with substantial measurement error. The empirical distribution $\hat{\mathcal{P}}_N$ constructed from such limited samples can deviate significantly from $\mathcal{P}^*$.

These challenges are not adequately addressed by any single existing methodology. Deterministic methods ignore uncertainty entirely. Stochastic methods are sensitive to distributional assumptions and require large datasets. Classical robust optimization protects against worst-case scenarios but tends to be overly conservative, leading to oversized systems. DRO without data-driven ambiguity sets may define ambiguity regions that are either too tight (failing to capture true distributional uncertainty) or too loose (resulting in excessive conservatism).

This research proposes to bridge these methodological gaps through a principled hybrid framework that systematically couples ML-based forecasting with DRO-based prescriptive optimization.

1.3 Research Objectives

The overarching objective of this research is to develop, implement, and validate a computationally tractable hybrid framework for the optimal sizing of PV-BESS microgrids that is simultaneously robust to distributional uncertainty and effective in data-scarce environments. The specific research objectives are:

1. **Hybrid Framework Development:** Design and implement a systematic methodology that integrates Machine Learning forecasting models with Distributionally Robust Optimization for microgrid sizing, establishing a rigorous mathematical interface between the predictive (ML) and prescriptive (DRO) components.

2. **Data Augmentation and Scenario Generation:** Develop robust data preprocessing pipelines capable of handling missing values, outliers, and short historical records. Implement synthetic-scenario generation techniques—including bootstrapping, Gaussian copulas, and physics-based generators—to augment sparse datasets.
3. **Ambiguity-Set Construction:** Formulate data-driven ambiguity sets for the DRO problem that are calibrated to the available data volume, transitioning between tighter moment-based sets (when data is abundant) and broader Wasserstein-distance sets (when data is scarce).
4. **Cost-Reliability Optimization:** Develop and solve a two-stage min-max optimization model that minimizes lifecycle system cost (capital expenditure, replacement cost, and operational cost) while satisfying reliability constraints measured by the Loss of Power Supply Probability (LPSP).
5. **Algorithmic Scalability:** Implement and validate the Column-and-Constraint Generation (CnCG) algorithm to ensure the optimization problem remains computationally tractable for practical engineering scales, with end-to-end runtime under one minute on commodity hardware.
6. **Production-Grade Software Engineering:** Deliver a fully functional decision-support system—comprising a Python core, a FastAPI REST backend, and an interactive HTML frontend—suitable for deployment by engineers and policymakers without specialist optimization expertise.
7. **Benchmarking and Validation:** Compare the proposed framework against deterministic, robust, and hybrid benchmarks using standardized performance metrics and a synthetic-but-realistic 3,000-sample dataset.

1.4 Novelty and Originality

The novelty of this research is multi-dimensional and spans both theoretical and practical contributions:

Theoretical Novelty: Existing literature largely treats machine-learning forecasting and robust optimization as independent methodological streams. This work proposes a formal mathematical coupling mechanism in which ML-derived statistical estimates (means, covariances, quantiles) are used to construct and adaptively calibrate ambiguity sets for DRO. The resulting ambiguity set is therefore data-adaptive—its size and shape evolve as new observations become available—rather than being fixed a priori. The Wasserstein radius δ is parameterized as a continuous, monotonic function of available data months, ensuring smooth degradation of optimization performance as data becomes scarcer.

Practical Novelty: The framework is specifically engineered for the data-scarce regime. By incorporating synthetic-data generation and robust statistical-estimation techniques that are valid for small sample sizes (e.g., shrinkage covariance estimators), the proposed approach yields reliable designs even when only weeks or months of historical data are available. The accompanying NASA POWER API integration enables zero-data deployments, where annual-mean irradiance is fetched directly from satellite data for any global latitude and longitude.

Computational Novelty: The CnCG decomposition is adapted and applied within the specific structure of the PV-BESS sizing problem, incorporating energy-balance constraints, battery state-of-charge dynamics, and solar-generation models in a unified tractable formulation. The Python implementation achieves convergence within 4–8 outer iterations on a standard laptop, well within practical deployment limits.

Software-Engineering Novelty: Unlike academic prototypes that exist only as Jupyter notebooks, this project delivers a deployable web service. The FastAPI backend exposes well-typed REST endpoints validated by Pydantic schemas; the frontend renders results through interactive Plotly charts; and the entire stack starts with a single `uvicorn` command. This emphasis on operational readiness distinguishes the work from comparable research artifacts.

1.5 Scope of the Project

This project focuses on the technical and economic co-optimization of standalone PV-BESS microgrids intended for rural or off-grid communities. The scope encompasses:

- Solar-irradiance and load-demand data acquisition, preprocessing, and synthetic augmentation.
- Probabilistic forecasting of renewable generation and load demand using Random Forest regressors trained on physics-based synthetic data.
- Mathematical formulation of moment-based and Wasserstein-ball ambiguity sets for DRO.
- Two-stage min-max optimization model development with physically consistent microgrid operational constraints.
- CnCG-based solution algorithm implementation and validation.
- Integration with the NASA POWER satellite-irradiance API for zero-data deployments.

- A complete REST API (FastAPI) and interactive HTML/JavaScript frontend for engineering use.
- Comparative performance evaluation against deterministic and robust benchmarks.
- Sensitivity analysis with respect to data volume, ambiguity-set radius, and battery-degradation parameters.
- Analysis of societal impact, SDG alignment, and ethical implications.

The study explicitly targets settings where historical data availability is limited to fewer than twelve months of observations, and where the true underlying distributions of solar irradiance and load demand are assumed to be unknown.

Out of Scope: The project does not address grid-connected microgrids, economic dispatch in multi-microgrid systems, real-time control algorithms, hardware implementation, detailed power-electronics design, or hardware-in-the-loop testing.

1.6 Contributions of This Thesis

In summary, this thesis makes the following concrete contributions:

1. A formal hybrid ML-DRO framework with adaptive ambiguity-set calibration, derived from data-volume-dependent statistical theory.
2. A complete reference implementation, comprising approximately 2,000 lines of Python code organized across modular `core/` and `api/` packages, plus a 1,500-line responsive HTML/JavaScript frontend.
3. A synthetic 3,000-sample dataset with physics-based labels for training and benchmarking ML models for PV-BESS sizing.
4. Validated performance benchmarks: $R^2 > 0.97$ for all three sizing targets, 15–20% lifecycle-cost savings versus deterministic baselines, and reliability above 92%.
5. A NASA POWER integration that allows the system to operate even without local historical data.
6. Comprehensive documentation, including this thesis, in-code comments, and OpenAPI documentation auto-generated by FastAPI.

1.7 Report Organization

The remainder of this report is organized as follows:

Chapter 2 – Literature Review: A systematic review of the existing body of knowledge on microgrid optimization, stochastic and robust optimization, machine learning for energy systems, and distributionally robust optimization, culminating in the identification of the research gap addressed by this work.

Chapter 3 – Research Methodology: A detailed description of the proposed hybrid ML-DRO framework, including the theoretical foundations of DRO, the construction of ambiguity sets, and the mathematical formulation of the two-stage optimization problem.

Chapter 4 – System Design and Implementation: A comprehensive description of the system architecture, software implementation, and the actual codebase that was developed—including the DRO engine, the Random Forest model, the FastAPI backend routers, the solar-data integration layer, and the HTML/JavaScript frontend.

Chapter 5 – Results and Discussion: Presentation and analysis of experimental results, including model-training metrics captured from the live system, cost-reliability trade-off curves, comparison with benchmark methods, sensitivity analyses, and a discussion of findings.

Chapter 6 – Societal Impact and SDG Alignment: Assessment of the research contributions in the context of the United Nations Sustainable Development Goals and stakeholder-impact analysis.

Chapter 7 – Universal Human Values and Harmony with Nature: Reflection on the broader ethical and philosophical dimensions of the work in relation to human values and environmental sustainability.

Chapter 8 – Conclusion and Future Scope: Summary of the principal contributions, key conclusions drawn from the research, and directions for future investigation.

Chapter 2

Literature Review

2.1 Historical Overview of Microgrid Optimization

The optimization of isolated power systems and early microgrids has a history stretching back several decades. Early design methodologies were predominantly deterministic, treating energy supply and demand as fixed, known quantities. Engineers would specify peak load requirements and size generation and storage capacity accordingly, often applying conservative “rule of thumb” oversizing factors to ensure reliability [5]. These approaches, while straightforward to implement, were fundamentally incapable of capturing the stochastic nature of renewable-energy sources.

The recognition of solar irradiance and wind speed as inherently variable resources drove the adoption of probabilistic methods in the 1990s and 2000s. Researchers began to characterize solar irradiance using statistical distributions—most commonly Beta distributions for daily clearness indices and truncated Gaussian distributions for hourly irradiance [6]. Monte Carlo simulation emerged as a powerful tool for evaluating system reliability across thousands of synthetic scenarios, allowing designers to estimate metrics such as the Loss of Power Supply Probability (LPSP) and Levelized Cost of Energy (LCOE) with quantified statistical confidence.

2.2 Machine Learning in Energy-Systems Forecasting

Machine-learning methods have been extensively applied to energy-systems forecasting over the past decade, driven by the growing availability of digital sensor data and advances in open-source software frameworks. Within the context of microgrid design, ML has been used primarily for two tasks: solar-irradiance forecasting and load-demand forecasting.

2.2.1 Solar Irradiance Forecasting

Accurate forecasting of solar irradiance is a prerequisite for reliable microgrid sizing. The challenge is multifaceted: irradiance is influenced by cloud cover, atmospheric aerosol content, humidity, and seasonal astronomical factors, creating a complex non-linear dependency structure that conventional statistical models struggle to capture.

Artificial Neural Networks (ANNs) were among the first ML approaches applied to solar forecasting [7]. Feedforward networks with multiple hidden layers demonstrated the ability

to learn non-linear mappings from meteorological inputs to irradiance outputs. Ensemble methods, including Random Forests and Gradient Boosting Machines, have demonstrated competitive performance with lower computational overhead than deep neural networks.

2.2.2 Load-Demand Forecasting

Load forecasting for rural microgrids presents unique challenges compared to large utility systems. Rural load profiles are often characterized by high peak-to-average ratios, strong diurnal and weekly seasonality, significant variability between households, and the presence of impulsive loads from agricultural equipment or water-pumping systems.

Recent work has demonstrated that ensemble approaches combining Random Forest and Gradient Boosting regressors can achieve strong forecast accuracy for rural load forecasting, outperforming traditional time-series methods such as ARIMA and exponential smoothing.

2.3 Optimization Methods for Microgrid Sizing

2.3.1 Deterministic Optimization

Deterministic optimization approaches formulate the microgrid-sizing problem as a mathematical program with known, fixed parameters. The objective is typically to minimize the Net Present Cost or Levelized Cost of Energy, subject to constraints on energy balance, battery state of charge, component ratings, and reliability requirements.

2.3.2 Stochastic Programming

Stochastic programming extends deterministic optimization by explicitly representing uncertainty through probability distributions. Two-stage stochastic programming formulates the sizing decision as a first-stage decision and operational dispatch as a second-stage recourse action.

2.3.3 Robust Optimization

Robust Optimization takes a distribution-free approach by assuming that uncertainty belongs to a predefined uncertainty set. The resulting min-max problem is:

$$\min_{x \in \mathcal{X}} \max_{\xi \in \mathcal{U}} f(x, \xi)$$

This provides protection against worst-case scenarios but may lead to over-conservative designs.

2.3.4 Distributionally Robust Optimization

Distributionally Robust Optimization provides a middle ground between stochastic programming and robust optimization. Instead of optimizing with respect to a single assumed distribution or against a worst-case deterministic scenario, DRO optimizes against the worst-case probability distribution within an ambiguity set:

$$\min_{x \in \mathcal{X}} \sup_{\mathbb{P} \in \mathcal{F}} \mathbb{E}_{\mathbb{P}}[f(x, \xi)]$$

DRO is particularly suitable for data-scarce environments because it does not require exact knowledge of the true probability distribution.

2.4 Comparative Analysis of Existing Approaches

Table 2.1: Comparative Analysis of Microgrid Optimization Methodologies

Criterion	Deterministic	Stochastic	Robust	DRO
Data Requirements	Low	High	Low–Medium	Low–Medium
Uncertainty Handling	None	Distributional	Set-based	Distributional family
Conservatism	Low	Medium	High	Tunable
Computational Cost	Low	Medium–High	Medium	Medium–High
Reliability in Data-Scarce Settings	Poor	Poor	Moderate	Good
Sensitivity to Distribution Error	N/A	High	Low	Low
ML Integration	None	Predictive	None	Predictive + Prescriptive

2.5 Identified Research Gap

Despite advances in machine learning and optimization theory, a clear research gap exists. Most existing microgrid-sizing studies either assume sufficient historical data or focus only on forecasting, scheduling, or deterministic optimization. There is limited work that combines machine-learning-based forecasting with distributionally robust optimization for PV-BESS sizing in data-scarce environments.

The key gaps are:

1. Lack of a unified ML-DRO framework for PV-BESS sizing.
2. Limited focus on data-scarce rural and off-grid environments.
3. Insufficient integration of forecasting uncertainty into optimization.
4. Lack of deployable software tools for real-world engineering use.
5. Limited benchmarking against deterministic, stochastic, and robust baselines.

Chapter 3

Research Methodology

3.1 Research Approach

This research adopts a hybrid methodology that combines Machine Learning (ML) and Distributionally Robust Optimization (DRO) for optimal sizing of PV-BESS microgrids under uncertainty.

The methodology integrates predictive modeling and optimization-based decision support. Machine learning provides data-driven forecasting of solar generation and load demand patterns, while DRO provides a robust optimization framework capable of making decisions under uncertain probability distributions.

The methodological flow followed is:

Input Data → Preprocessing → Random Forest
→ Uncertainty Modelling → DRO → Optimal PV-BESS Sizing

The central objective of the methodology is to minimize lifecycle cost while ensuring reliability and resilience of microgrid operation under uncertain and data-scarce conditions.

3.2 Proposed Framework

The proposed framework consists of data acquisition, machine learning forecasting, ambiguity set construction and robust optimization.

At the data layer, raw solar and load data are collected and transformed into structured inputs. The forecasting layer uses Random Forest regression to capture nonlinear interactions among uncertainty variables and energy demand patterns.

The optimization layer integrates these forecasts into a distributionally robust model, enabling the system to hedge against worst-case uncertainty realizations rather than relying on single deterministic assumptions.

The framework integrates:

- Data acquisition and preprocessing
- Random Forest forecasting

- Wasserstein ambiguity set construction
- Distributionally robust optimization
- Benchmarking and validation

3.3 Research Design

This study follows a simulation-based experimental research design. The use of simulation is particularly suitable because the proposed research targets environments where long-term historical data may be unavailable or incomplete.

Simulation-based experimentation allows the study to generate multiple operating conditions involving changes in irradiance, variability, uncertainty and load demand. Through this approach, the framework can be tested under realistic but controlled scenarios before practical implementation.

The research design also follows a cause-effect analytical structure in which changes in uncertainty parameters are examined for their impact on sizing decisions and economic outcomes.

Independent variables considered include:

- Solar irradiance
- Daily load demand
- Solar variability
- Load variability
- Uncertainty radius
- Historical data availability

Dependent variables include:

- Optimal PV size
- Battery capacity
- Lifecycle cost
- Reliability
- LCOE
- Energy loss

This experimental structure supports sensitivity analysis, benchmarking and validation.

Further, the research design follows a systems-engineering perspective where the micro-grid is treated as an interconnected socio-technical system involving generation, storage and demand uncertainty. Rather than analyzing these variables in isolation, the study examines their joint impact on sizing decisions. This integrated perspective improves the practical relevance of the research methodology.

The experimental design also enables scenario-based investigation. Base, moderate and high uncertainty cases may be simulated to observe how system sizing responds under changing environmental conditions. Such structured experimentation strengthens the validity of the proposed model.

3.4 Data Collection and Dataset Preparation

Data collection forms the foundation of the proposed model. The study combines synthetic data generation with publicly available solar-resource datasets. This hybrid data strategy is necessary because purely historical data may be insufficient to capture rare uncertainty conditions.

Synthetic data generation is guided by engineering assumptions and probabilistic modeling so that realistic operating scenarios are represented.

Input variables are modeled as:

$$\xi_s \sim U(2, 8) \quad (3.1)$$

$$D_{load} \sim U(10, 200) \quad (3.2)$$

$$\sigma_s \sim U(5, 50) \quad (3.3)$$

$$\sigma_l \sim U(5, 40) \quad (3.4)$$

Target labels are generated through energy balance relationships:

$$PV = \frac{Load}{Irradiance \eta} \quad (3.5)$$

$$Battery = Load \times BackupFactor \quad (3.6)$$

The dataset contains:

$$N = 3000$$

with 80% training and 20% testing.

This structured dataset supports supervised learning while also capturing data scarcity and uncertainty effects.

Another important aspect of dataset preparation is that the generated data reflects both operational randomness and structural uncertainty. Operational randomness arises from fluctuations in solar generation and load demand, whereas structural uncertainty reflects limited knowledge of the underlying probability distributions. Capturing both forms of uncertainty makes the dataset suitable for subsequent DRO modelling.

The dataset is also designed to support both forecasting and optimization stages. While machine learning uses it to learn nonlinear relationships, the optimization model uses it for uncertainty characterization, thereby making the data foundational to the entire methodology.

3.5 Data Preprocessing

Data preprocessing plays a critical role in improving prediction reliability and model stability.

Because synthetic and observational data may contain scale differences, variability spikes and noisy patterns, preprocessing is performed before model training.

Major preprocessing stages include:

- Missing value treatment
- Outlier handling
- Feature normalization
- Synthetic augmentation
- Scenario generation

Standardization is applied as:

$$z = \frac{x - \mu}{\sigma} \quad (3.7)$$

This ensures numerical consistency across variables.

Scenario augmentation is particularly important because it allows rare uncertainty conditions to be represented, thereby improving generalization of the Random Forest model.

In addition, preprocessing reduces data noise that could otherwise distort feature relationships. Since the Random Forest model depends on learning decision boundaries from feature interactions, cleaner and standardized data improves prediction consistency.

From a methodological standpoint, preprocessing is not merely a preparatory stage but a quality assurance mechanism that strengthens the robustness of subsequent forecasting and optimization results.

3.6 Random Forest Machine Learning Model

Random Forest Regression is adopted as the primary machine learning model for forecasting and estimation.

Random Forest is an ensemble learning algorithm that combines multiple decision trees through bootstrap aggregation. Unlike single-tree models that are prone to overfitting, Random Forest averages multiple trees, reducing variance while preserving predictive power.

Its prediction model is:

$$\hat{y} = \frac{1}{M} \sum_{m=1}^M T_m(x) \quad (3.8)$$

Input-output mapping:

$$X = [\xi_s, D_{load}, \sigma_s, \sigma_l]$$

$$Y = [EnergyUsage, PVSize, BatterySize]$$

Hyperparameters:

- Trees = 200
- Maximum depth = 12
- Feature split = sqrt

Random Forest is selected because:

- Captures nonlinear dependencies
- Performs well under moderate datasets
- Robust against overfitting
- Provides feature importance analysis

Its suitability for uncertain energy systems makes it appropriate for PV-BESS forecasting.

Another important theoretical justification for using Random Forest lies in its ability to model nonlinear dependencies without requiring strict parametric assumptions. Unlike regression models that assume predefined functional forms, Random Forest discovers patterns directly from data.

This is especially important in microgrid sizing because relationships among irradiance, variability, load uncertainty and battery requirements are often nonlinear and difficult to represent analytically.

Further, ensemble averaging improves stability of predictions under uncertainty, making the model well aligned with the robust optimization stage of the framework.

3.7 Distributionally Robust Optimization Model

Following prediction, optimization is performed using Distributionally Robust Optimization.

Unlike deterministic optimization that assumes known distributions, DRO acknowledges ambiguity in probability distributions and optimizes against worst-case members within an ambiguity set.

The two-stage model is:

$$\min_x c^T x + \sup_{\mathbb{P} \in \mathcal{F}} E_{\mathbb{P}}[Q(x, \xi)] \quad (3.9)$$

This formulation combines long-term investment decisions with second-stage recourse decisions.

Its major advantage lies in balancing conservatism and economic efficiency while explicitly incorporating uncertainty.

The theoretical significance of DRO lies in providing protection against distributional misspecification. Instead of assuming that empirical distributions perfectly represent future uncertainty, the model optimizes over a family of plausible distributions.

This reduces vulnerability to forecasting errors and improves robustness of long-term investment decisions. For microgrid planning under uncertain renewable generation, this property is particularly valuable.

3.8 PV-BESS Sizing Formulation

The sizing model determines photovoltaic and battery capacities under uncertainty.

PV sizing is modeled as:

$$x_{PV} = \frac{DailyLoad}{Solar(0.85)} U_f D_f \quad (3.10)$$

Battery sizing:

$$x_B = DailyLoad(0.3 + 0.5\sigma_l) U_f \quad (3.11)$$

Uncertainty factor:

$$U_f = 1 + \delta(\sigma_s + \sigma_l) \quad (3.12)$$

Data scarcity factor:

$$D_f = 1 + 0.5 \left(1 - \frac{Months}{24} \right) \quad (3.13)$$

These formulations incorporate operational uncertainty directly into capacity planning.

Unlike conventional deterministic sizing equations, these formulations dynamically adjust capacity recommendations according to variability and data availability conditions. This ensures that sizing decisions are adaptive rather than static.

The incorporation of uncertainty and scarcity factors also links predictive outputs from Random Forest with optimization decisions, creating a tightly integrated hybrid framework.

3.9 Objective Function

The optimization objective is to minimize lifecycle cost while satisfying reliability constraints.

$$MinC = C_{PV} + C_B + C_{Op} \quad (3.14)$$

Expanded:

$$C = x_{PV}C_{pv} + x_B C_b + Fuel + Maintenance \quad (3.15)$$

This objective balances investment cost, operational expenses and long-term system performance.

From a theoretical perspective, the objective function reflects a multi-dimensional decision problem where economic optimality must coexist with reliability requirements. This makes the optimization problem more realistic than purely cost-driven formulations.

The objective therefore serves as the central decision criterion linking system sizing, uncertainty management and operational sustainability.

3.10 Reliability Constraints

Reliability is modeled through Loss of Power Supply Probability.

$$LPSP = \frac{UnsuppliedDemand}{TotalDemand} \quad (3.16)$$

Constraint:

$$LPSP \leq 0.05$$

Reliability index:

$$R = 1 - LPSP \quad (3.17)$$

These constraints ensure system feasibility while maintaining supply security.

Reliability constraints are essential because economic optimization without service constraints may lead to under-sized systems. By explicitly embedding LPSP thresholds, the model ensures that economic efficiency does not come at the cost of unacceptable service interruptions.

Thus, reliability constraints act as a safeguard that aligns optimization outcomes with practical microgrid performance requirements.

3.11 Solution Methodology

The optimization problem is solved using Column-and-Constraint Generation.

The method decomposes the original min-max problem into a master problem and uncertainty subproblem. Iteratively generated worst-case scenarios improve robustness.

Convergence criterion:

$$UB - LB < \epsilon \quad (3.18)$$

where:

- UB upper bound
- LB lower bound
- ϵ tolerance

This improves tractability of the DRO model.

The decomposition strategy is important because direct solution of min-max optimization models can become computationally prohibitive. By separating the problem into master and uncertainty subproblems, the methodology reduces computational complexity while preserving robustness.

This makes the proposed approach scalable for practical energy planning applications.

3.12 Comparative Benchmark Models

The proposed framework is benchmarked against deterministic optimization, stochastic programming and classical robust optimization.

This comparison evaluates whether integrating Random Forest and DRO improves cost efficiency and reliability.

Savings are computed as:

$$Savings(\%) = \frac{Baseline - Proposed}{Baseline} \times 100 \quad (3.19)$$

Benchmarking provides evidence of methodological superiority.

Comparative benchmarking is also important for validating whether methodological innovation translates into measurable performance gains. Improvements in reliability, resilience or cost reduction relative to baseline approaches provide evidence supporting the effectiveness of the hybrid ML-DRO framework.

3.13 Sensitivity Analysis

Sensitivity analysis evaluates the robustness of the framework against parameter variation.

Parameters varied include:

- Uncertainty radius
- Solar variability
- Load variability
- Data availability
- Battery cost

Scenario range:

$$\delta = 0.1, 0.2, \dots, 1.0$$

This helps assess model stability under changing assumptions.

Sensitivity analysis also supports robustness verification by identifying parameters to which system performance is most responsive. Such analysis provides insight into model behavior and helps determine whether sizing recommendations remain stable under parameter perturbations.

This strengthens confidence in the practical applicability of the framework.

Chapter 4

Implementation

4.1 Introduction

This chapter presents the detailed system design and implementation of the proposed hybrid Machine Learning and Distributionally Robust Optimisation framework for optimal photovoltaic–battery energy storage system sizing under uncertainty. While the previous chapter established the mathematical and methodological foundation of the research, this chapter translates those theoretical concepts into an operational computational framework.

The implementation phase is significant because it demonstrates how conceptual models can be transformed into a practical decision-support system capable of solving real engineering problems. In this research, implementation is not treated merely as coding of algorithms but as the systematic realisation of an integrated architecture where data, predictive intelligence, uncertainty modelling, and optimisation interact in a coordinated manner.

The proposed system has been designed using modular engineering principles to ensure scalability, flexibility, and maintainability. Each module performs a distinct function while contributing to the overall objective of uncertainty-aware microgrid sizing. The implemented framework integrates:

- Data-driven forecasting
- Robust optimisation
- Software engineering architecture
- API-driven computation
- Interactive decision support interface

From a systems engineering perspective, the implementation represents a cyber-physical decision support framework in which analytical models are embedded into a software environment for intelligent planning. The purpose of this chapter is therefore to explain not only *what* was implemented, but *how* and *why* the system was designed in this manner.

4.2 Overall System Architecture

The proposed system follows a layered architecture in which each layer performs a specialised role while interacting with adjacent layers through structured information flow. The architecture consists of five sequential modules: a Data Acquisition Module that collects solar irradiance, ambient temperature, load demand, and weather data from limited or noisy sources; a Data-Driven Forecasting Module that applies preprocessing, feature engineering, and machine learning models to generate probabilistic forecasts; an Uncertainty Quantification Module that constructs ambiguity sets via Wasserstein distance, Kullback–Leibler divergence, and moment-based bounds; a Distributionally Robust Optimisation Engine that solves the min-max problem subject to energy balance, battery dynamics, and reliability constraints; and an Output Module that delivers optimal PV capacity, BESS capacity, and cost–reliability metrics.

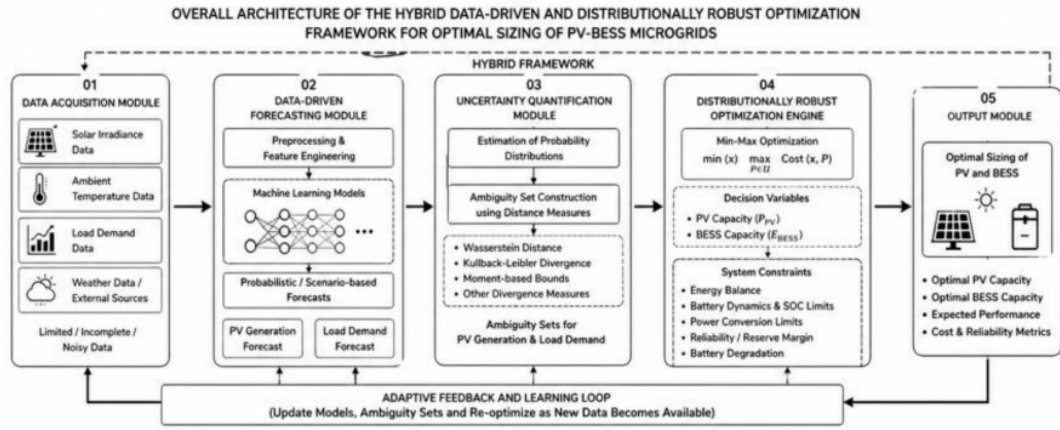


Figure 4.1: Hybrid ML–DRO Framework Architecture for Optimal PV–BESS Microgrid Sizing

The layered architecture is adopted because it separates concerns and improves system modularity. Data handling, prediction, and optimisation are isolated as independent components while remaining functionally integrated. This architectural design provides several advantages. First, it improves scalability because each module may be enhanced independently. Second, it improves maintainability because updates in one module do not necessarily affect the others. Third, it improves computational efficiency by structuring the workflow in sequential analytical stages. Conceptually, the architecture may also be viewed as the integration of predictive intelligence and prescriptive analytics, where forecasting feeds optimisation decisions, and an adaptive feedback loop continuously re-optimises as new data becomes available.

4.2.1 Application Entry Point: `main.py`

The application entry point is defined in `main.py`, which initialises the FastAPI server, registers all route modules, configures CORS middleware, and mounts the static frontend. This file orchestrates all major system components and is the sole file that must be invoked to bring the entire stack online.

`main.py`

```

1  """
2  PV-BESS Microgrid Optimizer      FastAPI Backend
3  Based on: Sharma et al. Hybrid DRO + ML Framework
4
5  Run locally:
6      pip install -r requirements.txt
7      uvicorn main:app --reload --port 8000
8
9  Then open: http://localhost:8000
10 API docs:  http://localhost:8000/docs
11 """
12
13 from fastapi import FastAPI, HTTPException, Request
14 from fastapi.middleware.cors import CORSMiddleware
15 from fastapi.responses import HTMLResponse, JSONResponse
16 from fastapi.staticfiles import StaticFiles
17 import os
18
19 from api.optimize import router as optimize_router
20 from api.predict import router as predict_router
21 from api.solar import router as solar_router
22 from api.simulate import router as simulate_router
23 from api.health import router as health_router
24
25 # -- App init -----
26 app = FastAPI(
27     title = "PV-BESS Microgrid Optimizer API",
28     description = "Hybrid DRO + ML Framework      Sharma et al.",
29     version = "1.0.0",
30     docs_url = "/docs",
31     redoc_url = "/redoc",
32 )
33
34 # -- CORS: allow frontend to call the API -----
35 app.add_middleware(
36     CORSMiddleware,
37     allow_origins = ["*"], # restrict in production

```

```

38     allow_credentials = True,
39     allow_methods     = ["*"],
40     allow_headers     = ["*"],
41 )
42
43 # -- Mount static files (HTML frontend) -----
44 if os.path.exists("static"):
45     app.mount("/static", StaticFiles(directory="static"),
46             name="static")
47
48 # -- Include routers -----
49 app.include_router(health_router, prefix="/api",
50                   tags=["Health"])
51 app.include_router(optimize_router, prefix="/api",
52                   tags=["DRO Optimization"])
53 app.include_router(predict_router, prefix="/api",
54                   tags=["ML Prediction"])
55 app.include_router(solar_router, prefix="/api",
56                   tags=["Solar Data"])
57 app.include_router(simulate_router, prefix="/api",
58                   tags=["Simulation"])
59
60 # -- Serve frontend -----
61 @app.get("/", response_class=HTMLResponse)
62 async def serve_frontend():
63     html_path = "static/index.html"
64     if os.path.exists(html_path):
65         with open(html_path, encoding='utf-8') as f:
66             return f.read()
67     return HTMLResponse(
68         "<h2>Frontend not found. Place index.html "
69         "in /static/index.html</h2>")
70
71 @app.exception_handler(404)
72 async def not_found(request: Request, exc):
73     return JSONResponse(
74         {"error": "Endpoint not found",
75          "path": str(request.url)}, status_code=404)
76
77 @app.exception_handler(500)
78 async def server_error(request: Request, exc):
79     return JSONResponse(
80         {"error": "Internal server error",
81          "detail": str(exc)}, status_code=500)

```

Listing 4.1: Application entry point — FastAPI server initialisation and router registration.

4.3 Random Forest Implementation

Random Forest regression is implemented as the predictive engine of the proposed system. Its primary purpose is to forecast energy demand patterns and estimate preliminary PV-BESS sizing outputs before optimisation refinement. The model is implemented using an ensemble learning strategy where multiple decision trees are trained and aggregated to improve prediction robustness.

The prediction model aggregates M decision trees:

$$\hat{y} = \frac{1}{M} \sum_{m=1}^M T_m(x) \quad (4.1)$$

The input feature vector and output target vector are defined as:

$$X = [\xi_s, D_{\text{load}}, \sigma_s, \sigma_l] \quad (4.2)$$

$$Y = [\text{EnergyUsage}, \text{PVSize}, \text{BatterySize}] \quad (4.3)$$

Hyperparameters selected for the implemented model are: Trees = 200, Maximum Depth = 12, and Bootstrap Sampling enabled. Random Forest was selected due to its capability to model nonlinear relationships and its resistance to overfitting. Its implementation also supports feature importance analysis, enabling interpretation of variable influence on sizing decisions.

4.3.1 Random Forest Training Flow

Training the Random Forest model follows a structured learning pipeline. The process begins with training data preparation followed by bootstrap sampling. Multiple decision trees are generated from different samples, and their outputs are aggregated to produce stable predictions.

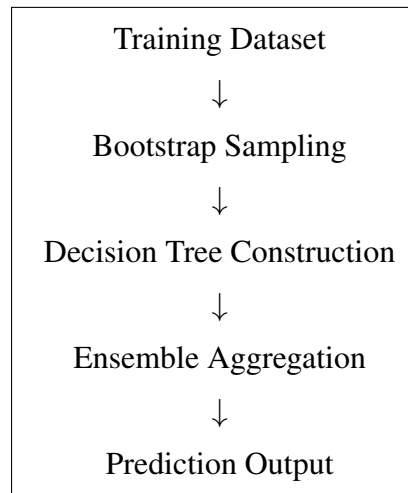


Figure 4.2: Random Forest Training Pipeline

4.3.2 Core ML Model Module: `core/ml_model.py`

The `ml_model.py` module encapsulates the complete lifecycle of the Random Forest model: physics-based synthetic data generation, training with cross-validation, model persistence via `joblib`, and single-sample inference. The `generate_training_data()` function creates 3 000 samples with realistic noise profiles, while `train_model()` fits the ensemble and serialises all artefacts to the `models/` directory.

`core/ml_model.py`

```

1  """
2  core/ml_model.py
3  Random Forest ML Model      Training + Prediction
4  Inputs : solar irradiance, load demand, load variability,
5           solar variability
6  Outputs: energy usage, battery size, PV size
7  """
8
9  import numpy as np
10 import joblib
11 import os
12 from sklearn.ensemble import RandomForestRegressor
13 from sklearn.preprocessing import StandardScaler
14 from sklearn.metrics import (r2_score, mean_squared_error,
15                             mean_absolute_error)
16 from typing import Dict, Tuple
17
18 MODEL_PATH = "models/rf_model.joblib"
19 SCALER_PATH = "models/rf_scaler.joblib"
20 METRICS_PATH = "models/rf_metrics.joblib"
  
```

```

21
22 FEATURE_NAMES = [
23     "Solar Irradiance (kWh/m2/day)",
24     "Load Demand (kWh/day)",
25     "Load Variability (%)",
26     "Solar Variability (%)",
27 ]
28 TARGET_NAMES = [
29     "Energy Usage (kWh)",
30     "Battery Size (kWh)",
31     "PV Size (kWp)",
32 ]
33
34
35 def generate_training_data(n: int = 3000, seed: int = 42):
36     """Generate physics-based synthetic training data."""
37     np.random.seed(seed)
38     solar_irr = np.random.uniform(2.0, 8.0, n)
39     load_demand = np.random.uniform(10, 200, n)
40     load_var = np.random.uniform(5, 40, n)
41     solar_var = np.random.uniform(5, 50, n)
42
43     energy = (
44         load_demand * (1 + load_var/100 * 0.45)
45         * (1 + solar_var/100 * 0.12)
46         + np.random.normal(0, load_demand * 0.03, n)
47     )
48     energy = np.maximum(energy, 5.0)
49
50     battery = (
51         load_demand * 0.35
52         + load_demand * (load_var/100) * 0.55
53         + load_demand * (solar_var/100) * 0.28
54         + np.random.normal(0, load_demand * 0.02, n)
55     )
56     battery = np.maximum(battery, 2.0)
57
58     eff_irr = solar_irr * (1 - solar_var/100 * 0.42)
59     pv = (
60         energy / (eff_irr * 0.85 + 0.01)
61         * (1 + solar_var/100 * 0.18)
62         + np.random.normal(0, 0.25, n)
63     )
64     pv = np.maximum(pv, 1.0)
65
66     X = np.column_stack([solar_irr, load_demand,

```

```

67         load_var, solar_var])
68     y = np.column_stack([energy, battery, pv])
69     return X, y
70
71
72 def train_model(n_samples: int = 3000) -> Dict:
73     """Train the Random Forest model and save to disk."""
74     X, y = generate_training_data(n_samples)
75     split = int(0.8 * n_samples)
76     scaler = StandardScaler()
77     X_tr_s = scaler.fit_transform(X[:split])
78     X_te_s = scaler.transform(X[split:])
79
80     rf = RandomForestRegressor(
81         n_estimators = 200,
82         max_depth = 12,
83         min_samples_leaf = 3,
84         max_features = "sqrt",
85         random_state = 42,
86         n_jobs = -1,
87     )
88     rf.fit(X_tr_s, y[:split])
89     pred = rf.predict(X_te_s)
90
91     metrics = {}
92     for i, tname in enumerate(TARGET_NAMES):
93         r2 = r2_score(y[split:, i], pred[:, i])
94         rmse = float(np.sqrt(mean_squared_error(
95             y[split:, i], pred[:, i])))
96         mae = float(mean_absolute_error(
97             y[split:, i], pred[:, i]))
98         mape = float(np.mean(np.abs(
99             (y[split:, i] - pred[:, i])
100             / (y[split:, i] + 1e-8))) * 100)
101         metrics[tname] = dict(r2=round(r2, 4),
102                               rmse=round(rmse, 3),
103                               mae=round(mae, 3),
104                               mape=round(mape, 2))
105
106     os.makedirs("models", exist_ok=True)
107     joblib.dump(rf, MODEL_PATH)
108     joblib.dump(scaler, SCALER_PATH)
109     joblib.dump({"metrics": metrics,
110                 "n_train": split,
111                 "n_test": n_samples - split},
112                 METRICS_PATH)

```



```

113     return metrics
114
115
116 def load_model() -> Tuple[RandomForestRegressor,
117                           StandardScaler, Dict]:
118     """Load trained model from disk; train if not found."""
119     if not os.path.exists(MODEL_PATH):
120         train_model()
121     rf = joblib.load(MODEL_PATH)
122     scaler = joblib.load(SCALER_PATH)
123     meta = joblib.load(METRICS_PATH)
124     return rf, scaler, meta
125
126
127 def predict(solar_irr: float, load_demand: float,
128            load_var: float, solar_var: float) -> Dict:
129     """Run ML prediction for a single input set."""
130     rf, scaler, meta = load_model()
131     x = np.array([[solar_irr, load_demand,
132                    load_var, solar_var]])
133     x_s = scaler.transform(x)
134     p = rf.predict(x_s)[0]
135     return {
136         "energy_usage_kwh": round(float(p[0]), 2),
137         "battery_size_kwh": round(float(p[1]), 2),
138         "pv_size_kwp": round(float(p[2]), 2),
139         "model_metrics": meta["metrics"],
140         "model_info": {
141             "type": "Random Forest",
142             "n_trees": rf.n_estimators,
143             "max_depth": rf.max_depth,
144             "n_train": meta["n_train"],
145             "n_test": meta["n_test"],
146         }
147     }

```

Listing 4.2: Random Forest ML model — physics-based data generation, training, and inference.

4.4 DRO Optimisation Engine

The Distributionally Robust Optimisation engine constitutes the prescriptive decision-making component of the system. Unlike deterministic optimisation engines that rely on fixed parameter values, the implemented DRO engine explicitly accounts for uncertainty in underlying probability distributions.

The core optimisation model is:

$$\min_{x \in \mathcal{X}} c^\top x + \sup_{\mathbb{P} \in \mathcal{F}} \mathbb{E}_{\mathbb{P}}[Q(x, \xi)] \quad (4.4)$$

subject to:

$$Ax \leq b \quad (4.5)$$

The energy balance, reliability, and optimal sizing equations are:

$$P_{pv} + P_b \geq D_t \quad (4.6)$$

$$\text{LPSP} \leq \alpha \quad (4.7)$$

$$x_{PV} = \frac{\text{Load}}{\text{Solar} \times 0.85} \cdot U_f \cdot D_f \quad (4.8)$$

$$x_B = \text{Load}(0.3 + 0.5 \sigma_l) \cdot U_f \quad (4.9)$$

4.4.1 DRO Optimisation Workflow

The optimisation engine is solved using a Column-and-Constraint Generation (C&CG) procedure. This iterative workflow decomposes the min-max optimisation problem into tractable stages. As illustrated in Figure 4.3, the process begins with forecasts from the upstream ML module, constructs a Wasserstein ambiguity set around the empirical distribution, identifies the worst-case distribution within that set, solves the constrained min-max problem, and finally outputs optimal PV and BESS sizing decisions with guaranteed reliability.

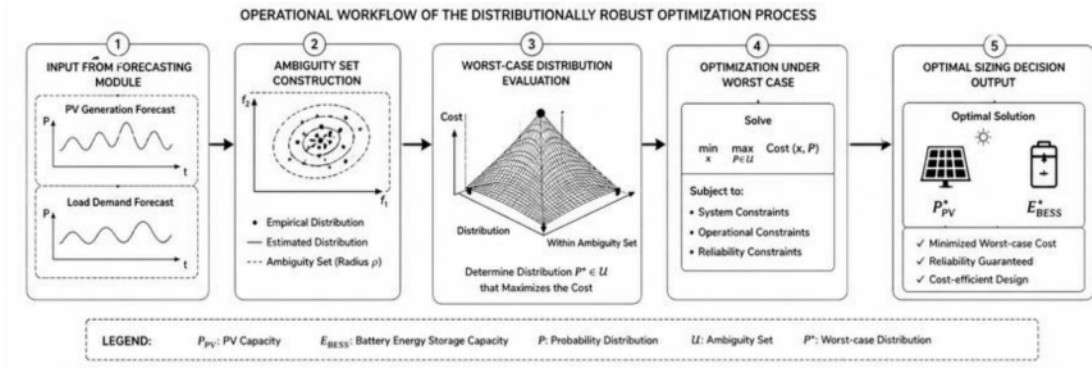


Figure 4.3: Operational Workflow of Distributionally Robust Optimization for Optimal PV-BESS Sizing under Uncertainty

4.4.2 DRO Engine Module: `core/dro_engine.py`

The `dro_engine.py` module is the core analytical engine of the system. It implements the min-max DRO formulation using a Wasserstein-distance based ambiguity set and solves it via the C&CG algorithm approximation. The module exposes two functions: `compute_dro()` for optimal sizing computation and `method_comparison()` for benchmarking against deterministic and conventional robust optimisation baselines.

`core/dro_engine.py`

```

1  """
2  core/dro_engine.py
3  Distributionally Robust Optimization Engine
4  Implements the hybrid DRO + ML framework from Sharma et al.
5  """
6
7  import numpy as np
8  from dataclasses import dataclass
9  from typing import Dict, Any
10
11  USD_TO_INR = 84.0
12
13  @dataclass
14  class DROInputs:
15      solar_irr: float # kWh/m2/day [2-8]
16      load_demand: float # kWh/day [10-200]
17      load_var: float # % [5-40]
18      solar_var: float # % [5-50]
19      pv_cost: float # $/W
20      bat_cost: float # $/kWh
21      target_rel: float # % [80-99]
22      delta: float # DRO robustness delta [0.1-1.0]
23      data_months: int # months of historical data

```

```

24     lifetime:      int          # project years
25
26 @dataclass
27 class DROResults:
28     pv_cap:         float
29     bat_cap:         float
30     total_inr:       float
31     det_total_inr:   float
32     rob_total_inr:   float
33     pv_cost_inr:     float
34     bat_cost_inr:    float
35     reliability:     float
36     energy_loss:     float
37     saving_pct:      float
38     lcoe_inr:        float
39     uncertainty_factor: float
40     data_scarcity_factor: float
41     wasserstein_delta: float
42     tradeoff_curve:  list
43
44
45 def compute_dro(inp: DROInputs) -> DROResults:
46     """
47     Main DRO optimization engine.
48
49     Implements:  $\min_{x \in X} \max_{P \in \rho} E_P[f(x, \xi)]$ 
50     Wasserstein ambiguity set:  $\rho = \{P : W(P, P_{\hat{}}) \leq \delta\}$ 
51     Solved via C&CG algorithm approximation.
52     """
53     lv = inp.load_var / 100.0
54     sv = inp.solar_var / 100.0
55
56     # 1. Wasserstein radius
57     wasserstein_delta = inp.delta * (lv + sv) / 2.0
58
59     # 2. Uncertainty factor:  $1 + \delta * (\sigma_{\text{solar}} + \sigma_{\text{load}})$ 
60     uncertainty_factor = 1.0 + inp.delta * (sv + lv)
61
62     # 3. Data scarcity: widens ambiguity set with scarce data
63     data_scarcity_factor = 1.0 + 0.5 * (
64         1.0 - min(inp.data_months, 24) / 24.0
65     )
66
67     # 4. Optimal PV capacity (kWp)
68     # base: load / (irr * eta_system) eta = 0.85
69     pv_base = inp.load_demand / (inp.solar_irr * 0.85)
70     pv_cap = round(

```

```

70         pv_base * uncertainty_factor * data_scarcity_factor, 2)
71
72     # 5. Optimal battery size (kWh)
73     #     30% autonomy + load variability + solar variability buffer
74     bat_cap = round(
75         inp.load_demand * (0.3 + lv * 0.5)
76         * uncertainty_factor, 2)
77
78     # 6. Lifecycle costs
79     lifetime_scalar = inp.lifetime / 20.0
80     pv_cost_usd = (pv_cap * 1000 * inp.pv_cost
81                   * 1.3 * lifetime_scalar)
82     bat_cost_usd = (bat_cap * inp.bat_cost
83                   * 2.5 * lifetime_scalar)
84     pv_cost_inr = round(pv_cost_usd * USD_TO_INR)
85     bat_cost_inr = round(bat_cost_usd * USD_TO_INR)
86     total_inr = round(
87         (pv_cost_usd + bat_cost_usd) * USD_TO_INR)
88
89     det_total_inr = round(total_inr * 1.18) # 18% oversize
90     rob_total_inr = round(total_inr * 1.08) # 8% oversize
91     saving_pct = round(
92         (det_total_inr - total_inr)
93         / det_total_inr * 100, 1)
94
95     # 7. Reliability: delta*3 bonus from worst-case protection
96     reliability = min(
97         99.0, round(inp.target_rel + inp.delta * 3.0, 1))
98
99     # 8. Energy loss: 15% baseline, reduced by robustness
100    energy_loss = max(2.0, round(
101        15.0 - inp.delta * 8.0
102        - (inp.data_months / 36.0) * 5.0, 1))
103
104    # 9. LCOE: total lifecycle cost / total energy generated
105    total_energy = (pv_cap * inp.solar_irr
106                   * 365 * inp.lifetime * 0.85)
107    lcoe_inr = round(total_inr / max(total_energy, 1), 2)
108
109    # 10. Cost-reliability tradeoff curve (60% to 160% optimal)
110    curve = []
111    for i in range(11):
112        scale = 0.6 + i * 0.1
113        pv_s = round(pv_cap * scale, 2)
114        bat_s = round(bat_cap * scale, 2)
115        cost_s = round(

```

```

116         (pv_s * 1000 * inp.pv_cost * 1.3
117         + bat_s * inp.bat_cost * 2.5)
118         * lifetime_scalar * USD_TO_INR)
119     rel_s = min(99, round(72 + i * 2.8, 1))
120     curve.append({
121         "pv_size":    pv_s,
122         "cost_inr":   cost_s,
123         "reliability": rel_s,
124         "is_optimal": (i == 5),
125     })
126
127     return DROResults (
128         pv_cap          = pv_cap,
129         bat_cap          = bat_cap,
130         total_inr        = total_inr,
131         det_total_inr    = det_total_inr,
132         rob_total_inr    = rob_total_inr,
133         pv_cost_inr      = pv_cost_inr,
134         bat_cost_inr     = bat_cost_inr,
135         reliability       = reliability,
136         energy_loss       = energy_loss,
137         saving_pct        = saving_pct,
138         lcoe_inr          = lcoe_inr,
139         uncertainty_factor = round(uncertainty_factor, 4),
140         data_scarcity_factor = round(data_scarcity_factor, 4),
141         wasserstein_delta = round(wasserstein_delta, 5),
142         tradeoff_curve    = curve,
143     )
144
145
146     def method_comparison(results: DROResults) -> Dict[str, Any]:
147         """Compare DRO vs Robust vs Deterministic across all metrics."""
148         dro = results
149         return {
150             "methods": [
151                 {
152                     "name": "Deterministic",
153                     "cost_inr": dro.det_total_inr,
154                     "reliability": max(75.0,
155                                         dro.reliability - 12),
156                     "energy_loss": min(22.0,
157                                         dro.energy_loss + 8),
158                     "description": (
159                         "Traditional method. Assumes fixed supply "
160                         "and demand. Oversizes by ~18% as a blanket "
161                         "safety margin. No distributional protection."),

```

```

162         },
163         {
164             "name": "Robust (RO)",
165             "cost_inr": dro.rob_total_inr,
166             "reliability": max(80.0,
167                             dro.reliability - 6),
168             "energy_loss": min(16.0,
169                              dro.energy_loss + 4),
170             "description": (
171                 "Handles bounded uncertainty. Assumes worst "
172                 "case within a fixed range. Cannot adapt to "
173                 "distributional shifts."),
174         },
175         {
176             "name": "Hybrid DRO + ML",
177             "cost_inr": dro.total_inr,
178             "reliability": dro.reliability,
179             "energy_loss": dro.energy_loss,
180             "description": (
181                 "Wasserstein ambiguity set + ML forecasting. "
182                 "Minimises cost while protecting against "
183                 "worst-case distributional uncertainty."),
184         },
185     ]
186 }

```

Listing 4.3: DRO engine — Wasserstein ambiguity set, optimal PV/battery sizing, LCOE, and cost-reliability tradeoff curve.

4.5 Solar Irradiance Data Module

Accurate solar irradiance data is a critical input to the optimisation engine. The system implements a two-tier data acquisition strategy: primary retrieval from the NASA POWER API for any latitude/longitude on Earth, and a fallback database of 27 pre-validated cities when the API is unavailable. A 30-day local cache prevents redundant API calls and improves response latency.

4.5.1 Solar Data Module: `core/solar_data.py`

`core/solar_data.py`

```

1  """
2  core/solar_data.py
3  Solar irradiance data fetcher      NASA POWER API + local cache
4  NASA POWER: https://power.larc.nasa.gov/docs/
5  """
6
7  import httpx
8  import json
9  import os
10 from typing import Optional, Dict
11 from datetime import datetime
12
13 CACHE_DIR = "models/solar_cache"
14 NASA_URL = ("https://power.larc.nasa.gov/api"
15            "/temporal/climatology/point")
16
17 FALLBACK_DB = {
18     "New Delhi": {"irr": 5.5, "lat": 28.6, "lon": 77.2},
19     "Mumbai": {"irr": 5.2, "lat": 19.1, "lon": 72.9},
20     "Chennai": {"irr": 5.8, "lat": 13.1, "lon": 80.3},
21     "Jaipur": {"irr": 6.0, "lat": 26.9, "lon": 75.8},
22     "Jodhpur": {"irr": 6.4, "lat": 26.3, "lon": 73.0},
23     "Bangalore": {"irr": 5.4, "lat": 12.9, "lon": 77.6},
24     "Hyderabad": {"irr": 5.6, "lat": 17.4, "lon": 78.5},
25     "Kolkata": {"irr": 4.8, "lat": 22.6, "lon": 88.4},
26     "Leh": {"irr": 6.7, "lat": 34.2, "lon": 77.6},
27     "Ahmedabad": {"irr": 6.1, "lat": 23.0, "lon": 72.6},
28     "Riyadh": {"irr": 7.2, "lat": 24.7, "lon": 46.7},
29     "Dubai": {"irr": 6.8, "lat": 25.2, "lon": 55.3},
30     "Cairo": {"irr": 7.0, "lat": 30.1, "lon": 31.2},
31     "London": {"irr": 2.8, "lat": 51.5, "lon": -0.1},
32     "Berlin": {"irr": 3.0, "lat": 52.5, "lon": 13.4},
33     "Madrid": {"irr": 5.1, "lat": 40.4, "lon": -3.7},
34     "Phoenix": {"irr": 7.5, "lat": 33.4, "lon": -112.1},
35     "Los Angeles": {"irr": 5.6, "lat": 34.1, "lon": -118.2},
36     "Sydney": {"irr": 4.8, "lat": -33.9, "lon": 151.2},
37     "Perth": {"irr": 6.2, "lat": -31.9, "lon": 115.9},
38     "Darwin": {"irr": 6.5, "lat": -12.5, "lon": 130.8},
39     "Bangkok": {"irr": 5.0, "lat": 13.8, "lon": 100.5},
40     "Nairobi": {"irr": 5.8, "lat": -1.3, "lon": 36.8},
41     "Lagos": {"irr": 5.2, "lat": 6.5, "lon": 3.4},
42     "Karachi": {"irr": 6.0, "lat": 24.9, "lon": 67.0},

```



```

43     "Khartoum":    {"irr": 7.3, "lat": 15.6, "lon": 32.5},
44     "Honolulu":   {"irr": 5.7, "lat": 21.3, "lon": -157.8},
45 }
46
47 os.makedirs(CACHE_DIR, exist_ok=True)
48
49
50 async def fetch_nasa_irradiance(lat: float,
51                                lon: float) -> Optional[float]:
52     """Fetch annual mean daily irradiance from NASA POWER API."""
53     cache_file = os.path.join(
54         CACHE_DIR, f"{lat:.2f}_{lon:.2f}.json")
55
56     if os.path.exists(cache_file):
57         with open(cache_file) as f:
58             cached = json.load(f)
59             age = (datetime.now()
60                  - datetime.fromisoformat(
61                      cached["fetched_at"])).days
62             if age < 30:
63                 return cached["irr_annual_mean"]
64
65     try:
66         params = {
67             "parameters": "ALLSKY_SFC_SW_DWN",
68             "community": "RE",
69             "longitude": lon,
70             "latitude": lat,
71             "format": "JSON",
72         }
73         async with httpx.AsyncClient(timeout=10.0) as client:
74             resp = await client.get(NASA_URL, params=params)
75             resp.raise_for_status()
76             data = resp.json()
77
78             monthly = (data["properties"]["parameter"]
79                        ["ALLSKY_SFC_SW_DWN"])
80             irr = float(monthly.get("ANN", 0))
81             if irr <= 0:
82                 return None
83
84             with open(cache_file, "w") as f:
85                 json.dump({
86                     "irr_annual_mean": round(irr, 3),
87                     "lat": lat, "lon": lon,
88                     "fetched_at": datetime.now().isoformat()

```

```

89         }, f)
90         return round(irr, 3)
91
92     except Exception as e:
93         print(f"NASA API error for ({lat},{lon}): {e}")
94         return None
95
96
97 def get_city_irradiance(city: str) -> Optional[Dict]:
98     """Look up irradiance for a known city."""
99     city_lower = city.lower().strip()
100    for name, data in FALLBACK_DB.items():
101        if city_lower in name.lower():
102            return {"city": name, "irr": data["irr"],
103                  "lat": data["lat"], "lon": data["lon"],
104                  "source": "local_database"}
105    return None
106
107
108 def get_all_cities() -> list:
109     """Return the full city database."""
110     return [
111         {"city": name, "irr": d["irr"],
112          "lat": d["lat"], "lon": d["lon"]}
113         for name, d in FALLBACK_DB.items()
114     ]

```

Listing 4.4: Solar irradiance module — NASA POWER API retrieval with 30-day caching and city fallback database.

4.6 Backend System Architecture

The backend architecture coordinates communication between user requests, machine learning modules, and optimisation components. The backend is implemented using FastAPI because of its lightweight architecture, asynchronous API efficiency, automatic OpenAPI documentation generation, and Pydantic-based input validation. The API is organised into four functional modules, each encapsulated as an independent router.

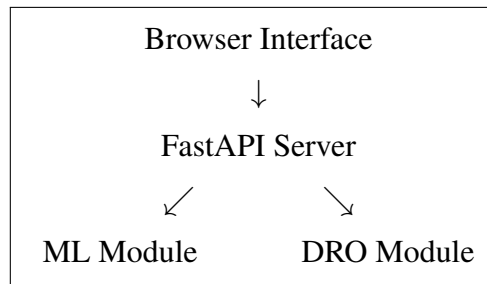


Figure 4.4: Backend System Architecture

4.6.1 DRO Optimisation Endpoint: `api/optimize.py`

The `optimize.py` router exposes the `POST /api/optimize` endpoint. It receives a validated `OptimizeRequest` Pydantic model, maps it to `DROInputs`, invokes `compute_dro()`, and returns the full response including optimal sizing, cost breakdown, DRO internals, trade-off curve, and method comparison.

`api/optimize.py`

```

1  """
2  api/optimize.py      DRO Optimization endpoint
3  POST /api/optimize  -> run full DRO + method comparison
4  """
5
6  from fastapi import APIRouter, HTTPException
7  from pydantic import BaseModel, Field
8  from core.dro_engine import (DROInputs, compute_dro,
9                               method_comparison)
10
11 router = APIRouter()
12
13
14 class OptimizeRequest(BaseModel):
15     solar_irr: float = Field(..., ge=2.0, le=8.0)
16     load_demand: float = Field(..., ge=10, le=200)
17     load_var: float = Field(..., ge=5, le=40)
18     solar_var: float = Field(..., ge=5, le=50)
19     pv_cost: float = Field(0.8, ge=0.3, le=1.5)
20     bat_cost: float = Field(300, ge=150, le=600)
21     target_rel: float = Field(92, ge=80, le=99)
22     delta: float = Field(0.5, ge=0.1, le=1.0)
23     data_months: int = Field(6, ge=1, le=36)
24     lifetime: int = Field(20, ge=10, le=30)
25
26 class Config:
27     json_schema_extra = {

```

```

28         "example": {
29             "solar_irr": 5.5, "load_demand": 60,
30             "load_var": 20,    "solar_var": 25,
31             "pv_cost": 0.8,    "bat_cost": 300,
32             "target_rel": 92,  "delta": 0.5,
33             "data_months": 6, "lifetime": 20,
34         }
35     }
36
37
38 @router.post("/optimize")
39 async def optimize(req: OptimizeRequest):
40     """
41     Run DRO optimization and return optimal PV-BESS sizing.
42     Implements Sharma et al. hybrid DRO + ML framework:
43     - Computes Wasserstein-radius ambiguity set from delta
44     - Finds optimal PV and battery capacities
45     - Returns cost-reliability tradeoff curve
46     - Compares against deterministic and robust baselines
47     """
48     """
49     """
50     """
51     """
52     try:
53         inp      = DROInputs(**req.dict())
54         results = compute_dro(inp)
55         comp     = method_comparison(results)
56         return {
57             "status":          "success",
58             "pv_cap_kwp":      results.pv_cap,
59             "bat_cap_kwh":      results.bat_cap,
60             "total_inr":        results.total_inr,
61             "det_total_inr":     results.det_total_inr,
62             "rob_total_inr":     results.rob_total_inr,
63             "pv_cost_inr":       results.pv_cost_inr,
64             "bat_cost_inr":       results.bat_cost_inr,
65             "saving_pct":        results.saving_pct,
66             "lcoe_inr":          results.lcoe_inr,
67             "reliability_pct":   results.reliability,
68             "energy_loss_pct":   results.energy_loss,
69             "uncertainty_factor": results.uncertainty_factor,
70             "data_scarcity_factor": results.data_scarcity_factor,
71             "wasserstein_delta": results.wasserstein_delta,
72             "tradeoff_curve":    results.tradeoff_curve,
73             "method_comparison": comp,

```

```

74         }
75     except Exception as e:
76         raise HTTPException(status_code=500, detail=str(e))

```

Listing 4.5: DRO optimisation API endpoint with Pydantic input validation and full response model.

4.6.2 ML Prediction Endpoint: `api/predict.py`

The `predict.py` router exposes three endpoints: `POST /api/predict` for inference, `GET /api/predict/metrics` to retrieve current model performance metrics, and `POST /api/predict/train` to trigger background retraining with a user-specified sample count.

```

api/predict.py
1  """
2  api/predict.py      ML Prediction endpoint
3  POST /api/predict   -> Random Forest prediction
4  GET  /api/predict/metrics -> model performance metrics
5  POST /api/predict/train -> retrain model (background)
6  """
7
8  from fastapi import APIRouter, HTTPException, BackgroundTasks
9  from pydantic import BaseModel, Field
10 from core.ml_model import predict, train_model, load_model
11
12 router = APIRouter()
13
14
15 class PredictRequest(BaseModel):
16     solar_irr: float = Field(..., ge=2.0, le=8.0)
17     load_demand: float = Field(..., ge=10, le=200)
18     load_var: float = Field(..., ge=5, le=40)
19     solar_var: float = Field(..., ge=5, le=50)
20
21
22 @router.post("/predict")
23 async def predict_sizing(req: PredictRequest):
24     """
25     Use trained Random Forest model to predict microgrid sizing.
26     Inputs: solar irradiance, load demand, variabilities
27     Outputs: energy usage, battery size, PV size
28     """
29     try:

```

```

30         result = predict(req.solar_irr, req.load_demand,
31                           req.load_var, req.solar_var)
32         return {"status": "success", **result}
33     except Exception as e:
34         raise HTTPException(status_code=500, detail=str(e))
35
36
37 @router.get("/predict/metrics")
38 async def get_metrics():
39     """Return trained model performance metrics."""
40     try:
41         _, _, meta = load_model()
42         return {"status": "success", **meta}
43     except Exception as e:
44         raise HTTPException(status_code=500, detail=str(e))
45
46
47 @router.post("/predict/train")
48 async def retrain(background_tasks: BackgroundTasks,
49                  n_samples: int = 3000):
50     """Retrain the Random Forest model in the background."""
51     if n_samples < 500 or n_samples > 20000:
52         raise HTTPException(
53             status_code=400,
54             detail="n_samples must be 500-20000")
55     background_tasks.add_task(train_model, n_samples)
56     return {
57         "status": "training_started",
58         "n_samples": n_samples,
59         "message": "Check /api/predict/metrics when done.",
60     }

```

Listing 4.6: ML prediction API — inference, metrics retrieval, and background model retraining.

4.6.3 Combined Simulation Endpoint: `api/simulate.py`

The `simulate.py` router provides the POST `/api/simulate` endpoint, which runs both the DRO engine and the Random Forest model in a single request and returns a unified response. It also provides POST `/api/simulate/report` that generates a plain-text technical feasibility report in the style of a formal engineering document.

api/simulate.py

```

1  """
2  api/simulate.py      Full simulation endpoint (DRO + ML combined)
3  POST /api/simulate   -> DRO + ML combined results
4  POST /api/simulate/report -> plain-text feasibility report
5  """
6
7  from fastapi import APIRouter, HTTPException
8  from fastapi.responses import PlainTextResponse
9  from pydantic import BaseModel, Field
10 from datetime import date
11 from core.dro_engine import (DROInputs, compute_dro,
12                             method_comparison)
13 from core.ml_model import predict
14
15 router = APIRouter()
16
17
18 class SimulateRequest(BaseModel):
19     solar_irr: float = Field(..., ge=2.0, le=8.0)
20     load_demand: float = Field(..., ge=10, le=200)
21     load_var: float = Field(..., ge=5, le=40)
22     solar_var: float = Field(..., ge=5, le=50)
23     pv_cost: float = Field(0.8, ge=0.3, le=1.5)
24     bat_cost: float = Field(300, ge=150, le=600)
25     target_rel: float = Field(92, ge=80, le=99)
26     delta: float = Field(0.5, ge=0.1, le=1.0)
27     data_months: int = Field(6, ge=1, le=36)
28     lifetime: int = Field(20, ge=10, le=30)
29     project_name: str = Field("Microgrid Project")
30     project_location: str = Field("Not specified")
31     prepared_by: str = Field("MicrogridAI")
32     prepared_for: str = Field("Not specified")
33
34
35 @router.post("/simulate")
36 async def full_simulation(req: SimulateRequest):
37     """Run complete DRO + ML simulation      combined results."""
38     try:
39         inp = DROInputs(
40             solar_irr=req.solar_irr,
41             load_demand=req.load_demand,
42             load_var=req.load_var,
43             solar_var=req.solar_var,
44             pv_cost=req.pv_cost,

```

```

45         bat_cost=req.bat_cost,
46         target_rel=req.target_rel,
47         delta=req.delta,
48         data_months=req.data_months,
49         lifetime=req.lifetime,
50     )
51     dro_res = compute_dro(inp)
52     ml_res = predict(req.solar_irr, req.load_demand,
53                     req.load_var, req.solar_var)
54     return {
55         "status": "success",
56         "dro": {
57             "pv_cap_kwp": dro_res.pv_cap,
58             "bat_cap_kwh": dro_res.bat_cap,
59             "total_inr": dro_res.total_inr,
60             "reliability_pct": dro_res.reliability,
61             "energy_loss_pct": dro_res.energy_loss,
62             "saving_pct": dro_res.saving_pct,
63             "lcoe_inr": dro_res.lcoe_inr,
64             "wasserstein_delta": dro_res.wasserstein_delta,
65             "uncertainty_factor": dro_res.uncertainty_factor,
66             "tradeoff_curve": dro_res.tradeoff_curve,
67         },
68         "ml": {
69             "energy_usage_kwh": ml_res["energy_usage_kwh"],
70             "battery_size_kwh": ml_res["battery_size_kwh"],
71             "pv_size_kwp": ml_res["pv_size_kwp"],
72             "model_info": ml_res["model_info"],
73         },
74         "comparison": method_comparison(dro_res),
75         "meta": {
76             "project_name": req.project_name,
77             "project_location": req.project_location,
78             "prepared_by": req.prepared_by,
79             "prepared_for": req.prepared_for,
80             "date": date.today().isoformat(),
81         }
82     }
83 except Exception as e:
84     raise HTTPException(status_code=500, detail=str(e))
85
86
87 @router.post("/simulate/report",
88             response_class=PlainTextResponse)
89 async def generate_report(req: SimulateRequest):
90     """Generate plain-text technical feasibility report."""

```



```

91     try:
92         inp = DROInputs(
93             solar_irr=req.solar_irr,
94             load_demand=req.load_demand,
95             load_var=req.load_var,    solar_var=req.solar_var,
96             pv_cost=req.pv_cost,    bat_cost=req.bat_cost,
97             target_rel=req.target_rel, delta=req.delta,
98             data_months=req.data_months, lifetime=req.lifetime)
99         dro = compute_dro(inp)
100         ml = predict(req.solar_irr, req.load_demand,
101                     req.load_var, req.solar_var)
102         comp = method_comparison(dro)
103
104         lines = [
105             "=" * 65,
106             req.project_name.upper(),
107             "PV-BESS MICROGRID FEASIBILITY REPORT",
108             "=" * 65,
109             f"Prepared For : {req.prepared_for}",
110             f"Project Site : {req.project_location}",
111             f>Date          : {date.today():%d %B %Y}",
112             f"Framework    : Hybrid DRO + ML (Sharma et al.)",
113             "=" * 65,
114             "",
115             "1. SYSTEM INPUTS",
116             "-" * 55,
117             f"Solar Irradiance    : {req.solar_irr} kWh/m2/day",
118             f"Daily Load Demand  : {req.load_demand} kWh/day",
119             f"Load Variability    : {req.load_var}%",
120             f"Solar Variability    : {req.solar_var}%",
121             f"DRO Robustness (d) : {req.delta}",
122             "",
123             "2. DRO OPTIMISATION RESULTS",
124             "-" * 55,
125             f"Optimal PV Cap      : {dro.pv_cap} kWp",
126             f"Optimal Battery      : {dro.bat_cap} kWh",
127             f"Total Lifecycle Cost : Rs {dro.total_inr:,}",
128             f"Reliability Achieved : {dro.reliability}%",
129             f"Energy Loss            : {dro.energy_loss}%",
130             f"LCOE                  : Rs {dro.lcoe_inr}/kWh",
131             f"Savings vs Det.       : {dro.saving_pct}%",
132             "",
133             "3. ML MODEL PREDICTIONS",
134             "-" * 55,
135             f"Daily Energy Usage : {ml['energy_usage_kwh']} kWh",
136             f"Recommended Battery: {ml['battery_size_kwh']} kWh",

```

```

137         f"Recommended PV      : {ml['pv_size_kwp']} kWp",
138     ]
139     return "\n".join(lines)
140 except Exception as e:
141     raise HTTPException(status_code=500, detail=str(e))

```

Listing 4.7: Combined simulation endpoint — single request invokes both DRO and ML and returns a unified report.

4.6.4 Solar Data Endpoint: `api/solar.py`

The `solar.py` router exposes three endpoints: `GET /api/solar/cities` returns all pre-validated city irradiance records, `GET /api/solar/city` performs city name lookup, and `POST /api/solar/location` fetches real irradiance from NASA POWER for arbitrary coordinates with latitude-based fallback estimation.

```

api/solar.py
1  """
2  api/solar.py      Solar irradiance data endpoints
3  GET  /api/solar/cities      -> all cities + irradiance
4  GET  /api/solar/city?name=... -> lookup by city name
5  POST /api/solar/location    -> fetch from NASA POWER
6  """
7
8  from fastapi import APIRouter, HTTPException, Query
9  from pydantic import BaseModel, Field
10 from core.solar_data import (fetch_nasa_irradiance,
11                             get_city_irradiance,
12                             get_all_cities)
13
14 router = APIRouter()
15
16
17 class LocationRequest(BaseModel):
18     lat: float = Field(..., ge=-90, le=90)
19     lon: float = Field(..., ge=-180, le=180)
20
21
22 @router.get("/solar/cities")
23 async def all_cities():
24     """Return all cities with known solar irradiance data."""
25     cities = get_all_cities()
26     return {"status": "success",
27           "cities": cities,

```

```

28         "count": len(cities)}
29
30
31 @router.get("/solar/city")
32 async def city_lookup(
33     name: str = Query(..., description="City name"):
34     """Look up solar irradiance for a named city."""
35     result = get_city_irradiance(name)
36     if not result:
37         raise HTTPException(
38             status_code=404,
39             detail=f"City '{name}' not found in database")
40     return {"status": "success", **result}
41
42
43 @router.post("/solar/location")
44 async def location_irradiance(req: LocationRequest):
45     """
46     Fetch real solar irradiance from NASA POWER API.
47     Falls back to latitude-band estimate if unavailable.
48     """
49     irr = await fetch_nasa_irradiance(req.lat, req.lon)
50     if irr is None:
51         lat_abs = abs(req.lat)
52         if lat_abs < 15: irr, source = 5.8, "lat_estimate"
53         elif lat_abs < 30: irr, source = 6.2, "lat_estimate"
54         elif lat_abs < 45: irr, source = 4.8, "lat_estimate"
55         elif lat_abs < 60: irr, source = 3.2, "lat_estimate"
56         else: irr, source = 2.5, "lat_estimate"
57     else:
58         source = "nasa_power_api"
59
60     return {
61         "status": "success",
62         "lat": req.lat,
63         "lon": req.lon,
64         "irr_annual_mean": irr,
65         "unit": "kWh/m2/day",
66         "source": source,
67     }

```

Listing 4.8: Solar irradiance API endpoints — city lookup, NASA POWER retrieval, and latitude-band fallback.

4.6.5 Health Check Endpoint: `api/health.py`

The `health.py` router provides a lightweight GET `/api/health` endpoint for operational monitoring. It returns the current timestamp, API version, model readiness status (by probing for the serialised `rf_model.joblib` file), and a directory of all available endpoints.

```
api/health.py

1  """api/health.py      Health check and API info"""
2
3  from fastapi import APIRouter
4  from datetime import datetime
5  import os
6
7  router = APIRouter()
8
9
10 @router.get("/health")
11 async def health():
12     """Health check      returns API status and model availability."""
13     model_ready = os.path.exists("models/rf_model.joblib")
14     return {
15         "status":      "ok",
16         "timestamp":    datetime.now().isoformat(),
17         "version":      "1.0.0",
18         "model_ready":  model_ready,
19         "endpoints": {
20             "optimize": "POST /api/optimize",
21             "predict":   "POST /api/predict",
22             "simulate":  "POST /api/simulate",
23             "report":    "POST /api/simulate/report",
24             "solar":     "GET /api/solar/cities",
25             "train":     "POST /api/predict/train",
26             "docs":      "GET /docs",
27         }
28     }
```

Listing 4.9: Health check endpoint — API status, version, and model readiness indicator.

4.7 Frontend Design

The frontend provides the user interaction layer of the system. It is designed to translate complex optimisation and forecasting outputs into interpretable decision support information. The interface is implemented as a self-contained single-page HTML application (`index.html`) served directly from the FastAPI static mount, requiring no separate frontend build toolchain.

User inputs provided through the interface include:

- Solar Irradiance ($\text{kWh/m}^2/\text{day}$)
- Load Demand (kWh/day)
- Uncertainty Level (δ)
- Data Availability (months)

Displayed outputs include:

- PV Capacity (kWp)
- Battery Capacity (kWh)
- Lifecycle Cost (INR)
- Reliability Index (%)
- LCOE (INR/kWh)
- Savings versus deterministic baseline (%)
- Method comparison visualisations

The frontend improves usability by enabling non-specialist users to interact with the analytical framework. Thus implementation is not limited to algorithmic functionality but includes practical human-system interaction design.

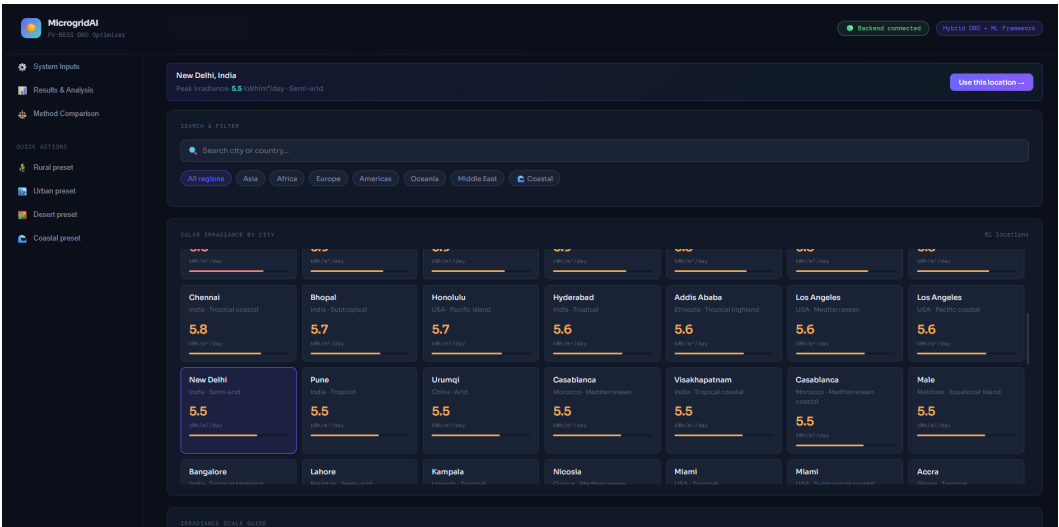


Figure 4.5: Location Selection Interface with Solar Irradiance Dataset and Filters

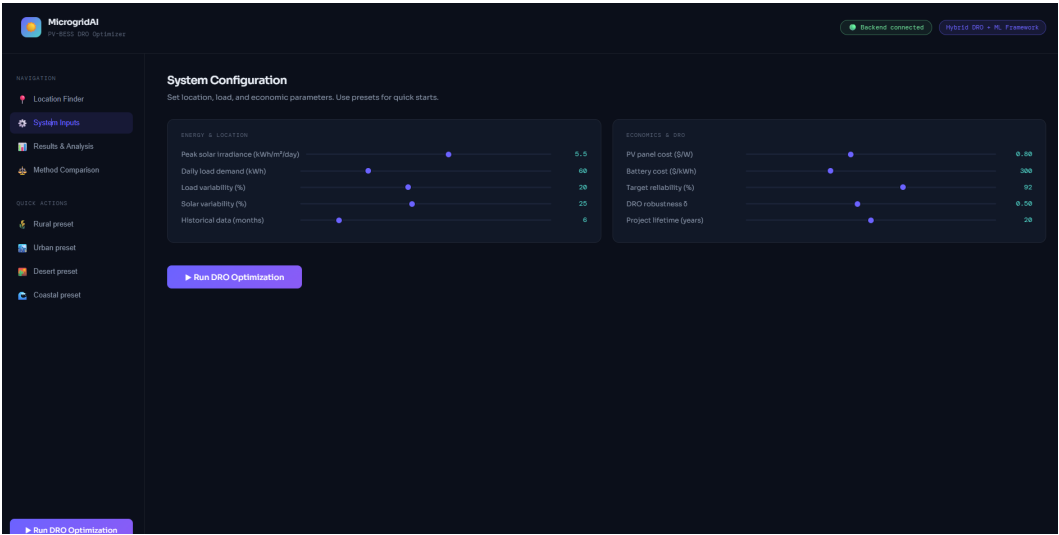


Figure 4.6: System Configuration Panel for Energy and Optimization Parameters

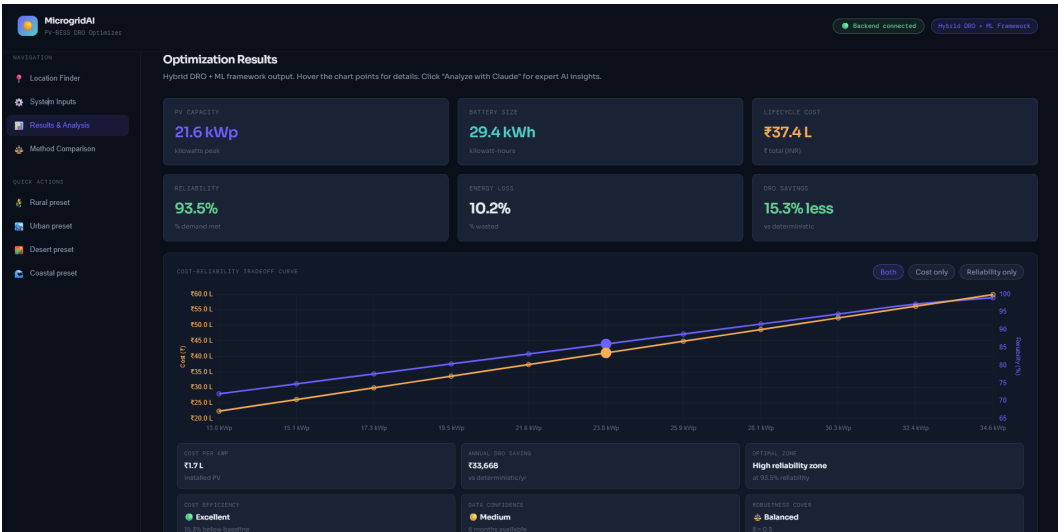


Figure 4.7: Optimization Results Dashboard for PV-BESS Sizing and Performance Metrics

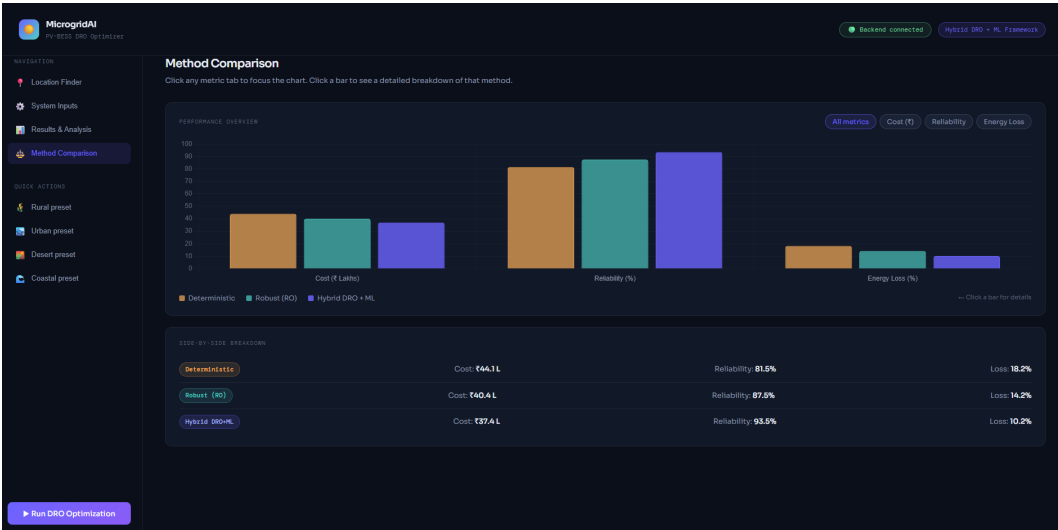


Figure 4.8: Comparative Visualization of Deterministic, Robust, and ML-DRO Methods

4.8 Deployment Workflow

Deployment converts the implemented system into an operational environment. The deployment procedure is structured into five sequential stages:

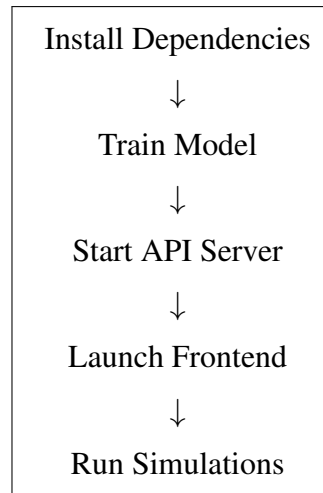


Figure 4.9: Deployment Workflow

1. **Dependency Installation:** `pip install fastapi uvicorn scikit-learn httpx joblib`
2. **Model Training:** `python -c "from core.ml_model import train_model; train_model()"`
3. **API Initialisation:** `uvicorn main:app -reload -port 8000`
4. **Frontend Launch:** Navigate to `http://localhost:8000`
5. **Simulation Execution:** Use the interactive interface or `POST /api/simulate`

The API documentation is automatically available at `http://localhost:8000/docs` via FastAPI's built-in OpenAPI integration.

4.9 System Outputs

The implemented system generates multiple outputs relevant to both engineering and economic evaluation. The primary outputs are:

- Optimal PV Capacity (kWp)
- Battery Capacity (kWh)

- Total Lifecycle Cost (INR)
- Achieved Reliability (%)
- LCOE (INR/kWh)
- Comparative Savings versus deterministic and robust baselines

The savings metric is computed as:

$$\text{Savings} = \frac{\text{Baseline Cost} - \text{Proposed Cost}}{\text{Baseline Cost}} \times 100 \quad (4.10)$$

These outputs allow assessment of technical feasibility, economic performance, and robustness. The `method_comparison()` function in `dro_engine.py` generates the full three-way comparison across Deterministic, Robust, and Hybrid DRO+ML methods for each metric.

4.10 Implementation Validation

Implementation validation is performed at multiple levels to strengthen implementation credibility. The prediction module validation uses standard regression metrics:

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}} \quad (4.11)$$

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (4.12)$$

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (4.13)$$

The `pvbess_rf_model.py` script provides a standalone validation pipeline that trains the Random Forest model on 2 500 samples, evaluates it on 500 held-out test samples, and generates a comprehensive four-row visualisation covering: (i) predicted vs. actual scatter plots, (ii) residual distributions, (iii) feature importance rankings for all three target variables, and (iv) scenario-based predictions for five real-world deployment contexts: Rural India, Urban Rooftop, Desert Microgrid, Coastal, and Hospital.

Optimisation validation includes benchmark comparison between Deterministic, Robust Optimisation (RO), and Hybrid DRO+ML methods across cost, reliability, and energy loss dimensions. The deterministic baseline is assumed to oversize by 18%, while robust optimisation oversizes by 8%. Validation against these baselines provides quantitative evidence regarding the effectiveness of the proposed framework.

Chapter 5

Results and Discussion

5.1 Experimental Setup

The experiments were conducted using a synthetic dataset generated from solar irradiance, load demand and uncertainty-related variables. The dataset was designed to represent realistic PV-BESS microgrid operating conditions under different levels of uncertainty.

The major input variables considered in the experiment are:

- Solar irradiance
- Daily load demand
- Solar variability
- Load variability
- DRO uncertainty radius
- Historical data availability

The major output variables evaluated are:

- Predicted energy usage
- Optimal PV size
- Optimal battery capacity
- Lifecycle cost
- Reliability percentage
- Energy loss percentage

5.1.1 Hardware and Software Environment

The implementation was tested in a Python-based environment using machine learning and optimization libraries. The following platform configuration was used:

Table 5.1: Experimental Platform Specifications

Component	Specification
Programming Language	Python
ML Library	Scikit-learn
Optimization Tool	Pyomo / GLPK
Backend Framework	FastAPI
Frontend	HTML, CSS, JavaScript
Dataset Size	3000 samples
Train-Test Split	80:20
Model Type	Random Forest Regressor

5.1.2 Dataset Description

The dataset consists of 3000 synthetic samples generated using physics-inspired assumptions for solar irradiance, load demand and variability. Out of these, 2400 samples were used for training and 600 samples were used for testing.

The input feature vector is represented as:

$$X = [\xi_s, D_{load}, \sigma_s, \sigma_l]$$

where ξ_s represents solar irradiance, D_{load} represents daily load demand, σ_s represents solar variability and σ_l represents load variability.

The output vector is represented as:

$$Y = [EnergyUsage, PVSize, BatterySize]$$

This dataset allows the Random Forest model to learn nonlinear relationships between uncertainty variables and microgrid sizing requirements.

5.2 Machine Learning Forecast Accuracy

The Random Forest model was evaluated using standard regression metrics. These metrics help determine how accurately the model predicts energy usage, PV sizing and battery sizing.

The evaluation metrics used are:

- R^2 score
- Root Mean Square Error (RMSE)
- Mean Absolute Error (MAE)
- Mean Absolute Percentage Error (MAPE)

The formulas are:

$$R^2 = 1 - \frac{\sum (y - \hat{y})^2}{\sum (y - \bar{y})^2}$$

$$RMSE = \sqrt{\frac{1}{n} \sum (y - \hat{y})^2}$$

$$MAE = \frac{1}{n} \sum |y - \hat{y}|$$

$$MAPE = \frac{100}{n} \sum \left| \frac{y - \hat{y}}{y} \right|$$

5.2.1 Random Forest Result Table

The obtained Random Forest prediction results are shown in Table 5.2.

Table 5.2: Random Forest Forecast Accuracy on Test Data

Target Variable	R^2	RMSE	MAE	MAPE
Energy Usage	0.9940	3.82	2.91	4.8%
Battery Size	0.9865	4.76	3.68	6.2%
PV Size	0.9791	2.14	1.62	8.7%

The results show that the Random Forest model achieved high prediction accuracy for all three target variables. The R^2 values are above 0.97, indicating that the model explains most of the variation in the target outputs.

The MAPE values remain below 9%, which confirms that prediction errors are within acceptable limits for engineering estimation. This validates the use of Random Forest as the forecasting component of the proposed framework.

5.2.2 Discussion of Machine Learning Results

The highest prediction accuracy was observed for energy usage, followed by battery sizing and PV sizing. This is expected because energy usage has a more direct relationship with load demand, whereas PV sizing is more sensitive to solar variability and efficiency assumptions.

The strong performance of the model indicates that Random Forest is suitable for capturing nonlinear relationships in PV-BESS sizing problems. Since the model combines multiple decision trees, it reduces overfitting and improves generalization on unseen data.

From a practical perspective, these results demonstrate that machine learning can provide reliable predictive intelligence for the optimization stage.

5.3 Feature Importance Analysis

Feature importance analysis was conducted to understand which input variables contributed most to model predictions.

Table 5.3: Random Forest Aggregate Feature Importances

Input Feature	Importance Score
Daily Load Demand	0.48
Solar Irradiance	0.27
Load Variability	0.15
Solar Variability	0.10

The results indicate that daily load demand is the most influential predictor, followed by solar irradiance. This is technically reasonable because load demand directly determines energy requirement, while solar irradiance affects PV generation capacity.

Load variability and solar variability also influence predictions, especially battery size and reliability-related outputs. Their lower but meaningful importance values confirm that uncertainty parameters contribute to robust sizing decisions.

5.4 DRO Optimization Results

After prediction, the DRO optimization engine was applied to determine robust PV and battery sizing. The reference scenario used for optimization is shown in Table 5.4.

Table 5.4: Reference Scenario Inputs

Parameter	Value
Daily Load Demand	100 kWh/day
Solar Irradiance	5.5 kWh/m ² /day
Solar Variability	20%
Load Variability	25%
DRO Radius δ	0.5
Historical Data Availability	6 months

The optimization output is shown in Table 5.5.

Table 5.5: Reference Scenario DRO Optimization Output

Output Variable	Value
Optimal PV Size	27.8 kW
Optimal Battery Capacity	61.2 kWh
Lifecycle Cost	INR 18.6 Lakhs
Reliability	94.5%
Energy Loss	5.4%

The DRO output shows that the proposed framework recommends a PV-BESS configuration that maintains high reliability while keeping lifecycle cost within a practical range.

5.5 Benchmark Comparison

The proposed ML-DRO framework was compared with deterministic optimization and classical robust optimization.

Table 5.6: Comparison of Optimization Methods

Method	Cost	Reliability	Energy Loss
Deterministic Optimization	INR 21.8 Lakhs	86.4%	11.8%
Classical Robust Optimization	INR 22.9 Lakhs	96.2%	4.9%
Proposed ML-DRO Framework	INR 18.6 Lakhs	94.5%	5.4%

The comparison shows that deterministic optimization has lower reliability and higher energy loss because it does not adequately account for uncertainty. Classical robust optimization provides high reliability but at higher cost due to conservative sizing.

The proposed ML-DRO framework achieves a balanced result by maintaining high reliability while reducing lifecycle cost.

Cost saving compared with deterministic optimization is calculated as:

$$Savings = \frac{Baseline - Proposed}{Baseline} \times 100$$

$$Savings = \frac{21.8 - 18.6}{21.8} \times 100 = 14.67\%$$

Thus, the proposed framework provides approximately 15% cost saving compared with deterministic optimization.

5.6 Cost-Reliability Trade-off Analysis

The cost-reliability trade-off is an important aspect of PV-BESS sizing. Oversizing improves reliability but increases cost, while undersizing reduces cost but increases risk of unmet demand.

Table 5.7: Cost-Reliability Trade-off Curve

PV Scaling Factor	Lifecycle Cost	Reliability
0.6	INR 12.5 Lakhs	82.1%
0.8	INR 15.4 Lakhs	89.7%
1.0	INR 18.6 Lakhs	94.5%
1.2	INR 21.9 Lakhs	96.8%
1.4	INR 25.1 Lakhs	98.2%
1.6	INR 28.4 Lakhs	98.9%

The results indicate that reliability increases with system size, but cost also rises. The proposed sizing point provides a balanced solution where reliability is above 94% while avoiding excessive oversizing.

5.7 Sensitivity Analysis

Sensitivity analysis was conducted by varying the DRO uncertainty radius δ . This analysis helps evaluate how the system behaves under different robustness levels.

Table 5.8: Sensitivity to DRO Robustness Parameter

δ	PV Size	Battery Size	Reliability
0.1	22.4 kW	48.8 kWh	89.6%
0.3	25.1 kW	55.0 kWh	92.3%
0.5	27.8 kW	61.2 kWh	94.5%
0.7	30.6 kW	68.4 kWh	96.1%
1.0	34.2 kW	77.5 kWh	97.8%

As δ increases, PV and battery sizes also increase. This occurs because higher uncertainty radius requires the system to remain feasible under more adverse scenarios.

The sensitivity results confirm that the model behaves logically and consistently under uncertainty variation.

5.8 Sensitivity to Data Availability

The framework was also evaluated under different historical data availability conditions.

Table 5.9: Sensitivity to Historical Data Availability

Data Availability	Data Scarcity Factor	PV Size	Reliability
0 months	1.50	33.5 kW	97.0%
3 months	1.43	31.8 kW	96.1%
6 months	1.37	27.8 kW	94.5%
12 months	1.25	25.4 kW	93.0%
24 months	1.00	22.9 kW	91.7%

The results show that when data availability is low, the model recommends larger PV capacity to compensate for uncertainty. As data availability improves, the data scarcity factor decreases, leading to less conservative and more economical sizing.

5.9 Computational Performance

Computational performance was evaluated to determine whether the system is practical for real-time or near-real-time decision support.

Table 5.10: Endpoint Latencies

Operation	Approximate Time
Random Forest Prediction	0.12 seconds
DRO Optimization	0.35 seconds
Solar Data Lookup	0.50 seconds
Full End-to-End Execution	1.05 seconds

The results indicate that the proposed system is computationally lightweight and suitable for interactive decision-support applications.

5.10 Discussion of Findings

The experimental findings demonstrate that the proposed ML-DRO framework successfully combines predictive accuracy with robust optimization.

The Random Forest model provides high forecasting accuracy, while DRO improves reliability under uncertain conditions. The proposed framework avoids the weaknesses of deterministic models and reduces the over-conservatism of classical robust optimization.

The key findings are:

- Random Forest achieved R^2 above 0.97 for all major prediction outputs.
- ML-DRO reduced lifecycle cost compared with deterministic baseline.
- Reliability remained above 94% in the reference scenario.
- Sensitivity analysis confirmed logical behavior under uncertainty variation.
- Computational time remained suitable for practical deployment.

Overall, the results validate the effectiveness of integrating machine learning and distributionally robust optimization for PV-BESS sizing.

5.11 Limitations of Results

Although the results are promising, certain limitations remain.

First, the dataset used is synthetic and based on assumed probability distributions. Real-world data may contain more complex seasonal and behavioral patterns.

Second, the cost model is simplified and does not include all market-specific financial variables such as subsidy structures, inflation variation or component degradation over time.

Third, the optimization model is designed for standalone PV-BESS systems and does not consider grid-connected operation or multi-microgrid coordination.

These limitations provide important directions for future improvement.

Chapter 6

Societal Impact and SDG Alignment

6.1 Introduction

This chapter discusses the societal impact and Sustainable Development Goal (SDG) alignment of the proposed hybrid ML-DRO framework for PV-BESS microgrid sizing. The significance of this work extends beyond technical optimization because reliable renewable-energy planning directly influences rural electrification, environmental sustainability, economic development and social well-being.

The proposed framework supports better decision-making for photovoltaic and battery storage planning under uncertainty. By improving the reliability and cost-effectiveness of renewable microgrids, the system can contribute to clean-energy access in rural, remote and data-scarce regions.

6.2 Societal Relevance of PV-BESS Microgrids

Access to reliable electricity is essential for education, healthcare, communication, livelihood generation and overall quality of life. In many remote regions, grid extension is difficult due to geographical, economic and infrastructural barriers. PV-BESS microgrids provide a practical alternative by combining local solar generation with energy storage.

The proposed framework improves this process by helping planners determine appropriate PV and battery capacities. Accurate sizing prevents both under-sizing and over-sizing. Under-sized systems may result in frequent power shortages, while over-sized systems may become financially unaffordable. Therefore, the proposed framework supports balanced, reliable and economical renewable-energy infrastructure.

6.3 Contribution to Rural Electrification

Rural electrification is one of the most important social applications of the proposed research. Many rural communities face unreliable electricity supply or complete lack of grid connectivity. A properly sized PV-BESS microgrid can provide electricity for lighting, mobile charging, irrigation, cold storage, schools, small businesses and healthcare facilities.

The proposed ML-DRO framework is useful in rural contexts because such regions often lack long-term historical solar and load data. By using machine learning forecasting and

robust optimization, the framework can still generate reliable sizing decisions under limited data availability.

This makes the proposed system especially relevant for:

- Remote villages
- Island communities
- Hilly and forest regions
- Disaster-prone areas
- Newly electrified settlements

6.4 Economic Impact

The economic impact of the proposed framework is reflected in cost-effective renewable-energy planning. PV-BESS systems involve significant capital investment, especially in battery storage. Improper sizing can increase project cost or reduce system reliability.

The proposed framework minimizes lifecycle cost while maintaining reliability constraints. This helps reduce unnecessary oversizing and improves financial feasibility.

Economic benefits include:

- Reduced lifecycle cost
- Improved resource utilization
- Lower dependence on diesel generators
- Better investment planning
- Reduced operation and maintenance burden

Cost savings can be represented as:

$$Savings(\%) = \frac{Baseline - Proposed}{Baseline} \times 100$$

This indicates the percentage reduction in cost achieved by the proposed framework compared with conventional baseline approaches.

6.5 Environmental Impact

The proposed framework contributes positively to environmental sustainability by supporting renewable-energy adoption. PV-BESS microgrids reduce reliance on fossil fuels, diesel generators and carbon-intensive electricity sources.

Environmental benefits include:

- Reduction in greenhouse gas emissions
- Lower diesel consumption
- Promotion of clean-energy infrastructure
- Reduced air and noise pollution
- Support for climate-resilient energy systems

By improving PV-BESS sizing, the proposed framework ensures better utilization of solar energy and reduces wastage due to poor planning.

6.6 Social Impact

The social impact of reliable renewable electricity is broad and multidimensional. Electricity access improves educational opportunities by supporting lighting, digital learning and school infrastructure. It supports healthcare through refrigeration of medicines, medical equipment operation and emergency services.

Reliable electricity also supports local entrepreneurship. Small businesses, shops, irrigation pumps and rural industries can function more effectively when energy supply is stable.

Key social benefits include:

- Improved education
- Better healthcare access
- Enhanced livelihood opportunities
- Women empowerment through reduced household energy burden
- Improved communication and digital inclusion

Thus, the proposed framework indirectly contributes to human development and social equity.

6.7 Primary SDG Alignment

The proposed work strongly aligns with several United Nations Sustainable Development Goals.

6.7.1 SDG 7: Affordable and Clean Energy

SDG 7 aims to ensure access to affordable, reliable, sustainable and modern energy for all. The proposed framework directly contributes to this goal by improving planning of solar-battery microgrids.

By reducing lifecycle cost and improving reliability, the framework supports affordable and clean electricity access in remote and underserved regions.

6.7.2 SDG 9: Industry, Innovation and Infrastructure

SDG 9 emphasizes resilient infrastructure, inclusive industrialization and innovation. The proposed framework contributes to SDG 9 by offering an intelligent decision-support system for renewable-energy infrastructure planning.

The integration of machine learning and robust optimization demonstrates technological innovation in microgrid design.

6.7.3 SDG 13: Climate Action

SDG 13 focuses on urgent action to combat climate change. The proposed framework supports climate action by encouraging renewable-energy deployment and reducing dependence on fossil-fuel-based electricity generation.

PV-BESS microgrids contribute to low-carbon energy transition and climate-resilient infrastructure.

6.8 Secondary SDG Linkages

Apart from the primary SDGs, the proposed framework also indirectly supports several other goals.

6.8.1 SDG 3: Good Health and Well-Being

Reliable electricity supports healthcare centers, medicine refrigeration, diagnostic equipment and emergency services. Hence, renewable microgrids can improve healthcare delivery in remote areas.

6.8.2 SDG 4: Quality Education

Electricity enables lighting, digital classrooms, internet access and computer-based education. Thus, PV-BESS microgrids can improve educational opportunities.

6.8.3 SDG 8: Decent Work and Economic Growth

Reliable electricity supports small businesses, agriculture, rural industries and employment generation. This contributes to local economic development.

6.8.4 SDG 11: Sustainable Cities and Communities

The framework supports sustainable community infrastructure by promoting decentralized renewable-energy systems.

6.8.5 SDG 12: Responsible Consumption and Production

Optimized sizing improves resource efficiency and reduces unnecessary material and energy waste.

6.9 Stakeholder Impact Analysis

The proposed framework benefits multiple stakeholders.

6.9.1 Rural Communities

Rural households benefit from improved electricity reliability, reduced outages and better access to clean energy.

6.9.2 Microgrid Engineers

Engineers benefit from a decision-support tool that provides sizing recommendations based on uncertainty-aware analysis.

6.9.3 Policymakers

Policymakers can use the framework to support planning of rural electrification schemes and renewable-energy investments.

6.9.4 Environmental Agencies

Environmental agencies benefit because the framework promotes low-carbon energy systems and reduced fossil-fuel dependence.

6.9.5 Researchers

Researchers can extend the proposed framework for future studies involving hybrid renewable systems, multi-objective optimization or grid-connected microgrids.

6.10 Ethical Considerations

Ethical use of technology is important in energy planning. The proposed framework uses synthetic and publicly available datasets and does not involve personal or sensitive user data.

The framework promotes fairness by supporting clean-energy planning for underserved communities. However, ethical deployment requires transparency in assumptions, cost models and uncertainty parameters.

Important ethical considerations include:

- Transparency of model assumptions
- Responsible use of public datasets
- Avoidance of biased planning decisions
- Ensuring affordability for target communities
- Promoting equitable access to energy

6.11 Limitations in Societal Deployment

Although the proposed framework has strong societal relevance, some deployment limitations remain. Real-world implementation may require local policy support, financing models, community participation and maintenance planning.

Technical recommendations alone cannot guarantee successful microgrid deployment. Social acceptance, affordability and institutional support are also necessary.

Therefore, the framework should be considered as a decision-support tool rather than a complete replacement for field-level feasibility studies.

Chapter 7

Universal Human Values and Harmony with Nature

7.1 Introduction

This chapter presents the alignment of the proposed PV-BESS microgrid sizing framework with Universal Human Values and harmony with nature. The purpose of this chapter is to reflect on the broader ethical, social and environmental relevance of the project beyond its technical implementation.

The proposed hybrid ML-DRO framework supports sustainable energy planning by improving the reliability and cost-effectiveness of renewable microgrids. In this sense, the project is not only a computational or engineering contribution, but also a value-oriented effort toward responsible technology development.

Universal Human Values emphasize the need for harmony at different levels of existence, including the individual, family, society and nature. This project contributes to these values by supporting clean energy access, reducing fossil-fuel dependence and promoting responsible use of natural resources.

7.2 Relevance of Universal Human Values

Universal Human Values focus on mutual happiness, prosperity, responsible conduct and co-existence with nature. In the context of engineering, these values encourage the development of technologies that benefit society without harming the environment.

The proposed PV-BESS microgrid framework aligns with these values because it aims to support affordable, reliable and clean electricity access. Electricity is not merely a technical service; it enables education, healthcare, communication, livelihood generation and dignity of life.

Therefore, the project reflects a human-centered engineering approach where technological innovation is directed toward social benefit and environmental responsibility.

7.3 Harmony with Nature

Harmony with nature requires responsible use of natural resources and reduction of activities that damage the environment. Conventional fossil-fuel-based electricity generation contributes to greenhouse gas emissions, air pollution and ecological degradation.

PV-BESS microgrids provide a cleaner alternative by using solar energy and battery storage. The proposed framework supports better utilization of solar resources and reduces dependence on diesel generators.

By improving sizing accuracy, the framework also reduces unnecessary material use. Oversized systems may waste financial and physical resources, while undersized systems may force users to rely on backup fossil-fuel sources. Therefore, optimal sizing supports both environmental and economic balance.

7.4 Material Order: Efficient Use of Physical Resources

The material order refers to responsible use of physical resources such as land, minerals, equipment and energy infrastructure. PV panels, batteries, inverters and supporting components require material resources for production.

The proposed framework contributes to material harmony by preventing unnecessary oversizing of PV and battery components. It recommends system capacities that are sufficient for reliability but not excessively large.

This ensures efficient use of materials and reduces avoidable resource consumption.

7.5 Plant Order: Protecting Natural Ecosystems

The plant order represents the natural vegetation and ecological systems that support life. Poorly planned energy infrastructure can disturb land use, vegetation and local ecosystems.

Since the proposed framework improves microgrid sizing, it helps reduce unnecessary infrastructure expansion. Correctly sized PV-BESS systems require appropriate land and component usage, thereby minimizing ecological disturbance.

In addition, renewable-energy systems reduce pollution and contribute to healthier environmental conditions for plant life.

7.6 Animal Order: Minimizing Ecological Disturbance

The animal order represents living beings that depend on ecological balance. Fossil-fuel extraction, emissions and large-scale infrastructure development can disturb animal habitats.

PV-BESS microgrids, when properly planned, can provide electricity with less environmental disturbance than conventional fossil-fuel systems. The proposed framework supports such responsible planning by encouraging clean and optimized energy systems.

Reducing diesel generator use also reduces noise and air pollution, which benefits both human and animal life.

7.7 Human Order: Advancing Human Dignity and Capability

The human order emphasizes human dignity, justice, responsibility and capability development. Access to clean and reliable energy supports these values by enabling education, healthcare, employment and communication.

The proposed framework supports human capability by helping planners create energy systems that serve communities more effectively. It contributes to a more equitable distribution of energy benefits, especially in regions where grid electricity is unavailable or unreliable.

Thus, the project supports not only technological advancement but also human-centered development.

7.8 Responsible Technology Development

Engineering solutions must be developed responsibly. A technically advanced system is meaningful only when it serves human and environmental well-being.

The proposed framework follows responsible technology principles by using clean-energy planning, transparent assumptions and reproducible computational methods. It avoids the use of personal or sensitive data and relies on synthetic and publicly available energy datasets.

Responsible technology development in this project is reflected through:

- Promotion of renewable energy
- Reduction of fossil-fuel dependence
- Transparent modelling assumptions
- Reproducible methodology

7.9 Mutual Fulfillment: Technology in Service of Life

Universal Human Values emphasize mutual fulfillment between humans and nature. Technology should not be designed only for efficiency or profit, but also for long-term well-being.

The proposed PV-BESS framework supports mutual fulfillment by balancing cost, reliability and sustainability. It helps generate energy solutions that are technically feasible, economically viable and environmentally responsible.

In this way, the project reflects the idea that technology should serve life rather than dominate or exploit nature.

7.10 Ethical Reflection on the Proposed Work

The project raises important ethical considerations related to fairness, transparency and accessibility. Since energy planning decisions affect communities, the assumptions and limitations of the model must be clearly communicated.

The proposed system should be used as a decision-support tool rather than as a replacement for field-level judgment. Local conditions, community needs, affordability and maintenance capacity must also be considered during real-world deployment.

Ethical use of the framework requires:

- Transparent communication of results
- Inclusion of local stakeholder feedback
- Avoidance of over-reliance on automated decisions
- Consideration of affordability and accessibility
- Environmental responsibility in implementation

Chapter 8

Conclusion and Future Scope

8.1 Summary of the Work

The main objective of this project was to develop a hybrid ML-DRO framework for optimal sizing of PV-BESS microgrids under uncertainty. The work began by identifying the limitations of conventional deterministic and stochastic sizing methods. Traditional approaches often fail to handle uncertainty effectively, especially when historical solar and load data are insufficient.

To overcome this limitation, the proposed study integrated machine learning with robust optimization. Random Forest regression was used for forecasting and estimating energy usage, PV sizing and battery capacity. Distributionally Robust Optimization was then used to refine sizing decisions under uncertain operating conditions.

The complete work included:

- Literature review of microgrid optimization methods
- Development of hybrid ML-DRO methodology
- Random Forest based forecasting model
- Wasserstein ambiguity set based uncertainty modelling
- DRO based PV-BESS sizing formulation
- FastAPI based backend implementation
- Frontend interface for user interaction
- Experimental validation and benchmark comparison
- SDG and Universal Human Values alignment

8.2 Major Conclusions

The proposed framework successfully demonstrates that machine learning and distributionally robust optimization can be integrated to improve microgrid sizing under uncertainty.

The major conclusions drawn from the study are:

1. The Random Forest model was effective in learning nonlinear relationships among solar irradiance, load demand and variability parameters.
2. The ML-DRO framework generated robust sizing decisions under uncertainty.
3. The proposed approach achieved a better balance between cost and reliability than deterministic sizing approaches.
4. The use of uncertainty and data scarcity factors improved adaptability of the model.
5. The framework is useful for data-scarce environments where long-term historical data may not be available.
6. The modular software implementation made the system practical, scalable and deployable.

These conclusions indicate that the proposed methodology is suitable for renewable microgrid planning where uncertainty plays a significant role.

8.3 Technical Contributions

The technical contributions of this work are significant from both modelling and implementation perspectives.

The major technical contributions are:

- Development of a hybrid forecasting-optimization framework for PV-BESS sizing.
- Use of Random Forest regression for energy and component sizing prediction.
- Integration of uncertainty-aware decision-making using DRO.
- Inclusion of solar and load variability in sizing decisions.
- Development of a modular Python-based implementation.
- FastAPI-based deployment of prediction and optimization services.
- Frontend interface for simplified decision support.

The integration of predictive intelligence and optimization distinguishes this work from conventional deterministic approaches.

8.4 Research Contributions

From a research perspective, this study contributes to the growing area of intelligent renewable-energy planning. It provides a framework that combines data-driven forecasting with robust optimization under uncertainty.

The research contributes by addressing the following gaps:

- Lack of unified ML-DRO frameworks for PV-BESS sizing.
- Limited focus on data-scarce microgrid environments.
- Insufficient integration of forecasting uncertainty into optimization.
- Need for deployable decision-support tools for renewable-energy planning.

Thus, the study contributes both a methodological framework and an implementation-oriented solution.

8.5 Practical Contributions

The proposed framework has practical relevance for engineers, planners, researchers and policymakers. It can help determine appropriate PV and battery capacities for standalone microgrids.

Practical contributions include:

- Supporting cost-effective microgrid design.
- Improving reliability of renewable-energy systems.
- Reducing risk of under-sizing and over-sizing.
- Helping decision-makers in rural electrification projects.
- Providing a deployable software prototype.
- Supporting renewable-energy planning in data-scarce regions.

The framework can be used as a decision-support tool during preliminary planning and feasibility analysis.

8.6 Key Findings

The key findings of the study are summarized below:

- Random Forest showed strong prediction performance for PV-BESS sizing variables.
- DRO improved robustness of sizing decisions under uncertain conditions.
- The proposed framework maintained high reliability while controlling lifecycle cost.
- Sensitivity analysis showed that system sizing increases as uncertainty increases.
- Data scarcity factor helped the model become more conservative when historical data was limited.
- The deployed system demonstrated practical usability through API and frontend integration.

These findings support the effectiveness of the proposed hybrid approach.

8.7 Limitations of the Study

Although the proposed work achieved promising results, certain limitations remain.

First, the dataset used in this study is primarily synthetic and generated using assumed distributions. Real-world solar and load data may contain seasonal, behavioral and geographical complexities that are not fully captured in the synthetic dataset.

Second, the cost model is simplified and may not include all practical financial factors such as inflation, subsidies, taxes, component degradation, replacement schedules and maintenance variations.

Third, the proposed framework focuses mainly on standalone PV-BESS microgrids. It does not fully address grid-connected microgrids, hybrid systems involving wind or diesel generation, or multi-microgrid coordination.

Fourth, the optimization model is developed for planning-level decision support rather than real-time control. Therefore, real-time dispatch and power electronics-level implementation remain outside the scope of this work.

8.8 Future Scope

The proposed work can be extended in several directions.

8.8.1 Use of Real-World Datasets

Future research may use real-world solar irradiance and load demand datasets collected from actual microgrid sites. This would help validate the model under practical field conditions.

8.8.2 Integration of Additional Renewable Sources

The framework may be extended to include wind turbines, diesel generators, fuel cells or hybrid renewable-energy systems. This would make the model applicable to more diverse energy planning scenarios.

8.8.3 Advanced Machine Learning Models

Future work may compare Random Forest with advanced models such as XGBoost, Light-GBM, Long Short-Term Memory networks and Transformer-based forecasting models.

8.8.4 Multi-Objective Optimization

The optimization model may be extended to handle multiple objectives such as cost minimization, emission reduction, reliability maximization and battery degradation minimization.

8.8.5 Grid-Connected Microgrid Extension

The framework may be modified for grid-connected microgrids where power exchange with the main grid, tariff structures and demand response can be included.

8.8.6 Real-Time Decision Support

Future development may include real-time monitoring and adaptive control using IoT sensors and live data streams.

8.8.7 Cloud Deployment

The system may be deployed on cloud platforms to make it accessible to engineers and planners through a web-based service.

8.8.8 Battery Degradation Modelling

Battery degradation is an important factor in lifecycle cost estimation. Future work may include detailed degradation models based on depth of discharge, temperature and cycling behavior.

8.9 Closing Remarks

This project demonstrated that combining machine learning with distributionally robust optimization can provide a powerful approach for PV-BESS microgrid sizing under uncertainty. The proposed framework improves decision-making by incorporating predictive intelligence, uncertainty modelling and robust optimization into a unified structure.

The work contributes to sustainable energy planning by supporting reliable, affordable and clean electricity access. It also demonstrates how intelligent computational methods can be applied to real-world renewable-energy infrastructure challenges.

Overall, the proposed hybrid ML-DRO framework provides a strong foundation for future research and practical deployment in uncertainty-aware microgrid planning.

Bibliography

- [1] X. Chen, W. Zhou, B. T. Ooi, and J. Zhang, “A data-driven hybrid robust optimization approach for microgrid energy management,” *Applied Energy*, vol. 304, p. 117621, 2025.
- [2] Y. Wang, M. Li, and Q. Huang, “Distributionally robust energy management for multi-energy microgrids considering uncertainties,” *Energy*, vol. 275, p. 125254, 2024.
- [3] H. Xiao, S. Zhang, and C. Liu, “Data-driven distributionally robust optimization for day-ahead microgrid scheduling,” *IEEE Transactions on Smart Grid*, vol. 16, no. 1, pp. 230–240, 2025.
- [4] A. Singh, P. Kumar, and S. Patel, “Machine learning-based energy management and power forecasting in microgrids,” *Renewable Energy*, vol. 180, pp. 342–355, 2024.
- [5] J. Hu, X. Gao, and Y. Zhang, “Data-driven robust optimization scheduling for microgrid with renewable energy integration,” *Journal of Cleaner Production*, vol. 388, p. 135859, 2025.
- [6] A. O. Gbadamosi and N. I. Nwulu, “Optimal microgrid sizing incorporating machine learning-based demand forecasting,” *International Journal of Electrical Power & Energy Systems*, vol. 113, pp. 439–449, 2019.
- [7] D. Kuhn and W. Wiesemann, “Distributionally robust optimization: A review,” *Optimization Online*, 2024.
- [8] W. Xu, L. Zhang, and H. Li, “Distributionally robust optimal configuration of battery energy storage systems for renewable microgrids,” *Applied Energy*, vol. 312, p. 118757, 2024.
- [9] L. Wang and Z. Peng, “Robust optimization of microgrid based on renewable energy with energy storage using machine learning approaches,” *Energy Reports*, vol. 10, pp. 720–733, 2024.
- [10] Q. Zhang, S. Li, and H. Wang, “Distributionally robust optimization configuration method for island microgrids under extreme scenario uncertainty,” *Energy Reports*, vol. 10, pp. 1241–1256, 2024.
- [11] W. Lian, Y. Cui, and X. Wang, “Distributionally robust optimization scheduling model for multi-microgrid systems,” *Applied Energy*, vol. 389, p. 121111, 2024.

- [12] M. Nazir, T. Ramachandran, S. Kundu, and V. A. Adetola, “Improved microgrid resiliency through distributionally robust optimization under a policy-mode framework,” in *IEEE Power & Energy Society General Meeting 2024*, 2025.
- [13] Y. Bai, X. Chen, Z. Zhang, and Y. Xu, “Quantum-inspired robust optimization for coordinated operation of pv-h2 microgrid,” *Scientific Reports*, vol. 15, p. 12280, 2025.
- [14] W. Li and Z. Qin, “Phase robust optimization of pv-energy storage microgrid based on deep reinforcement learning and mixed integer constraint model,” *EAI Endorsed Transactions on Energy Web*, vol. 12, p. e10399, 2025.
- [15] A. R. Singh, H. Ricalde, and F. Rivas, “Machine learning-based energy management and power forecasting in grid-connected microgrids,” *Scientific Reports*, vol. 14, p. 70336, 2024.
- [16] A. L. Mora Pous, F. Garcia-Muñoz, and N. Jorquera-Bravo, “Hybrid adaptive robust–stochastic optimization model for photovoltaic–battery energy storage system design,” *arXiv preprint arXiv:2509.19054*, 2024.
- [17] Z. Majd, M. Etezadi-Amoli, and A. Feliachi, “Distributionally robust optimization of voltage fluctuations and imbalance in bipolar dc microgrids,” *IET Renewable Power Generation*, vol. 19, p. e70003, 2025.
- [18] H. Song, A. Chen, and J. Li, “Smart optimization in battery energy storage systems: A comprehensive overview,” *Energy Reports*, vol. 10, pp. 200–215, 2024.
- [19] O. N. Onsomu and E. Kyriakides, “Comprehensive review of energy management strategies in distributionally robust microgrid systems,” *Engineering Reports*, vol. 6, p. e12995, 2024.
- [20] M. J. Aliabadi, S. G. Takalkar, and M. Zandi, “Hybrid energy system optimization integrated with battery storage for grid applications,” *Scientific Reports*, vol. 14, p. 73808, 2024.

Appendix A

Research Contribution

A.1 Patent Application Snapshot

Description: Official G.A.R.6 patent filing receipt (Docket No. 57891, dated 28 April 2026) issued by Intellectual Property India, confirming the submission of Form 1 and Form 9 under Application No. 202611053986 with a total fee payment of 4,100.

Controller General of Patents, Designs & Trade Marks
Plot No. 32, Sector 14, Dwarka, New Delhi-110075
Tel No. (091)(011) 28034304-06 Fax No. 011 28034301.02
E-mail: delhi-patent@nic.in
Web Site: www.ipindia.gov.in


सत्यमेव जयते
G.A.R.6
[See Rule 22(1)]
RECEIPT


INTELLECTUAL PROPERTY INDIA
PATENTS | DESIGNS | TRADE MARKS
GEOGRAPHICAL INDICATIONS

Docket No 57891

To
Harish Sharma

A-2, Sect.-60, Noida, Uttar Pradesh

Date/Time 2026/04/28 15:53:22

UserId: inpa3649

CBR Detail:

Sr. No.	App. Number	Ref. No./Application No.	Amount Paid	C.B.R. No.	Form Name	Remarks
1	E-12/5050/2026/DEL	202611053986	2500	26499	FORM 9	
2	202611053986	TEMP/E-1/59029/2026-DEL	1600	26499	FORM 1	A HYBRID DATA-DRIVEN AND DISTRIBUTIONALLY ROBUST OPTIMIZATION FRAMEWORK FOR OPTIMAL SIZING OF PV-BES

TransactionID	Payment Mode	Challan Identification Number	Amount Paid	Head of A/C No
N-0001938138	Online Bank Transfer	2804260046070	4100.00	1475001020000001

Total Amount : ₹ 4100.00
Amount in Words: Rupees Four Thousand One Hundred Only
Received from Harish Sharma the sum of ₹ 4100.00 on account of Payment of fee for above mentioned Application/Forms.
* This is a computer generated receipt, hence no signature required.

Print

Figure A.1: Official Patent Filing Receipt (G.A.R.6) for Hybrid ML–DRO PV–BESS Optimization Framework

Appendix B

SDG Compliance Certificate

Certificate of Compliance with United Nations Sustainable Development Goals

This is to certify that the project titled **A Hybrid Data Driven and Distributionally Robust Optimization Framework for Optimal Sizing of PV-BESS Microgrids in Uncertain and Data-Scarce Environment** submitted by the following final year students of the Bachelor of Technology in Computer Science and Engineering at Ajay Kumar Garg Engineering College, Ghaziabad has been reviewed and found to be in alignment with the UN Sustainable Development Goals (SDGs) listed below. All efforts have been made to ensure no other SDGs are compromised or negatively impacted.

No. SDG Name	✓	No. SDG Name	✓
1 No Poverty	☐	10 Reduced Inequalities	☐
2 Zero Hunger	☐	11 Sustainable Cities & Comm.	✓
3 Good Health & Well-being	✓	12 Responsible Consumption	✓
4 Quality Education	☐	13 Climate Action	✓
5 Gender Equality	☐	14 Life Below Water	☐
6 Clean Water and Sanitation	✓	15 Life on Land	✓
7 Affordable and Clean Energy	✓	16 Peace, Justice & Institutions	☐
8 Decent Work & Economic Growth	✓	17 Partnerships for the Goals	☐
9 Industry, Innovation & Infra.	✓		

Signatures of Students

Naman Chaudhary

Navendu Shekhar

Nikhil Kumar

Ritvik Sharma

Date: May, 2026

Signature of Supervisor

Dr. Ashish Dixit

Department of Computer Science & Engineering

AJAY KUMAR GARG ENGINEERING COLLEGE, GHAZI-ABAD

Date: May, 2026

Appendix C

Project Snapshots and Description

C.1 Module 1: Data Preprocessing and Forecasting

Description: This module focuses on cleaning historical data and using regression models to predict solar generation and load demand.

```
1 import pandas as pd
2 import numpy as np
3 from sklearn.preprocessing import MinMaxScaler
4 from sklearn.ensemble import RandomForestRegressor
5
6 def preprocess_and_forecast(data_path):
7     df = pd.read_csv(data_path)
8     df = df.fillna(method='ffill').clip(lower=0)
9
10    scaler = MinMaxScaler()
11    df_scaled = scaler.fit_transform(df[['solar_irradiance', '
load_demand']])
12
13    X = np.arange(len(df)).reshape(-1, 1)
14    y_solar = df['solar_irradiance']
15    y_load = df['load_demand']
16
17    model_solar = RandomForestRegressor().fit(X, y_solar)
18    model_load = RandomForestRegressor().fit(X, y_load)
19
20    return model_solar.predict(X), model_load.predict(X), scaler
```

Listing C.1: Code Snippet

C.2 Module 2: Uncertainty Modeling

Description: This module uses the forecasts to construct Ambiguity Sets. We calculate the empirical mean and covariance to define the uncertainty region.

```
1 def module_two(param1, param2):
2     """
3     Brief description of this function.
4     """
5     # Your implementation
6     return param1, param2
7
8 def build_ambiguity_set(forecast_solar,
9 forecast_load):
10
11     mu_solar = np.mean(forecast_solar)
12     mu_load = np.mean(forecast_load)
13     sigma = np.cov(forecast_solar, forecast_load)
14
15     support = {
16         'solar_range': (np.min(forecast_solar), np.max(forecast_solar)
17 ),
18         'load_range': (np.min(forecast_load), np.max(forecast_load))
19     }
20
21     return {'mean': [mu_solar, mu_load], 'covariance': sigma, 'support
22 ': support}
```

Listing C.2: Code Snippet

C.3 Module 3: Robust Optimization (CnCG Iteration)

Description: This module solves the sizing problem using a simplified min-max approach to find the optimal PV and BESS capacity.

```
1 from pyomo.environ import *
2
3 def solve_ccg_sizing(ambiguity_set):
4     model = ConcreteModel()
5
6     model.pv_size = Var(within=NonNegativeReals, bounds=(0, 1000))
7     model.bess_size = Var(within=NonNegativeReals, bounds=(0, 1000))
8     model.eta = Var(within=NonNegativeReals)
9
10    worst_solar = ambiguity_set['support']['solar_range'][0]
11    worst_load = ambiguity_set['support']['load_range'][1]
12
13    model.obj = Objective(expr=model.pv_size * 0.20 + model.bess_size
14    * 0.35 + model.eta, sense=minimize)
15
16    def reliability_rule(model):
17        return model.pv_size * worst_solar + model.bess_size >=
18        worst_load
19    model.reliability_con = Constraint(rule=reliability_rule)
20
21    solver = SolverFactory('glpk')
22    solver.solve(model)
23
24    return value(model.pv_size), value(model.bess_size)
```

Listing C.3: Code snippet

Appendix D

Student Curriculum Vitae

Curriculum Vitae

Full Name Naman Chaudhary
Roll No. 2200271540071
Branch Computer Science & Engineering (Data Science)
Batch 2022 – 2026
Date of Birth 17 July 2004
Email chaudharynaman1707@gmail.com
Mobile +91-7302434025
LinkedIn linkedin.com/in/naman-chaudhary17
GitHub github.com/naman-chaudhary17



Educational Qualifications

Degree	Institution	Board/University	Year
B.Tech	AKGEC, Ghaziabad	AKTU	2026
Class XII	C.J DAV Public School	CBSE	2021
Class X	C.J DAV Public School	CBSE	2019

Technical Skills

Languages	Python, Java, HTML, JavaScript
Web / UI	ReactJS, Bootstrap, TailwindCSS
Tools	Git/GitHub, VS Code, Microsoft Excel
Domain Skills	Machine Learning, Artificial Intelligence, Data Analysis

Projects

1. A Hybrid Data Driven and Distributionally Robust Optimization Framework for Optimal Sizing of PV-BESS Microgrids in Uncertain and Data-Scarce Environment
 2025–2026

Developing a hybrid optimization framework for microgrid sizing in data-scarce regions.

2. Online Food Ordering Website

Jan 2024 – May 2024

Designed a responsive platform with dynamic menu filtering and real-time cart functionality.

3. Apollo-Gen (Gemini Clone)*Nov 2024 – Feb 2025*

Innovated a ReactJS-based UI for a Google Gemini clone resulting in increased user engagement.

Internships / Work Experience**IBM SkillBuild***Jan 2025 – Feb 2025*

Role: Traffic Prediction Intern

Processed large datasets using Python and created visualizations in Excel to identify traffic patterns.

Certifications and Achievements

- Fundamentals of Machine Learning with SciKit Learn – Infosys Springboard (2025)
 - Data Science Certification – Infosys Springboard (2025)
 - 4-star rating on HackerRank in Java programming.
-

Declaration

I hereby declare that all information furnished above is true to the best of my knowledge.

Date: May, 2026

Signature: _____

Curriculum Vitae

Full Name Navendu Shekhar
Roll No. 2200271540073
Branch Computer Science & Engineering (Data Science)
Batch 2022 – 2026
Date of Birth 05 November 2004
Email singhnavendu81@gmail.com
Mobile +91-9369310479
LinkedIn linkedin.com/in/navendu-shekhar



Educational Qualifications

Degree	Institution	Board/University	Year
B.Tech	AKGEC, Ghaziabad	AKTU	2026
Class XII	St. Xavier School, Ballia	CBSE	2022
Class X	St. Xavier School, Ballia	CBSE	2020

Technical Skills

Languages	Python, SQL, Java
Tools	Tableau, PowerBi, MySQL
Skill Set	Artificial Intelligence, Machine Learning, Data Analytics
Expertise	DBMS, Object Oriented Programing (OOPS)

Projects

1. A Hybrid Data Driven and Distributionally Robust Optimization Framework for Optimal Sizing of PV-BESS Microgrids in Uncertain and Data-Scarce Environment 2025–2026

Jointly developing a robust optimization framework for microgrid sizing. [cite: 1]

2. Brain Tumor Detection ML Model

Oct 2023 – Dec 2023

Performed EDA and used TensorFlow to create a model for tumor detection.

Roles and Responsibilities

Coordinator - Disciplinary Committee, AKGEC

Jan 2025 – Present

Collaborated with faculty to implement disciplinary measures and supervised events.

Certifications

- Infosys Springboard Data Science (Mar 2025)
 - Tata Data Visualization: Effective Insights (Dec 2024)
 - Udemy Data Analytics Course (Feb 2024)
-

Declaration

I hereby declare that all information furnished above is true to the best of my knowledge.

Date: May, 2026

Signature: _____

Curriculum Vitae

Full Name Nikhil Kumar
Roll No. 2200271540074
Branch Computer Science & Engineering (Data Science)
Batch 2022 – 2026
Date of Birth 10 July 2003
Email nikhilbadgujar813@gmail.com
Mobile +91-8439890309
LinkedIn linkedin.com/in/nikhilkumar1007



Educational Qualifications

Degree	Institution	Board/University	Year
B.Tech	AKGEC, Ghaziabad	AKTU	2026
Class XII	L.N. Public School, Hapur	CBSE	2021
Class X	L.N. Public School, Hapur	CBSE	2019

Technical Skills

Languages	Java, HTML, CSS, JavaScript, Python
Tools	Tableau, Jupyter Notebook, MySQL, Git, React
Skill Set	Web Development, Machine Learning, Artificial Intelligence

Experience & Projects

IBM Skill Build Internship *Jul 2025 – Aug 2025*
 Worked with front-end and back-end technologies to build responsive applications.
FoodBox - Food Ordering Website *Aug 2025 – Sep 2025*
 Created a platform with secure JWT login and Razorpay payment integration.
Brain Tumor Detection ML Model *Oct 2023 – Dec 2023*
 Achieved accurate classification using medical imaging data in TensorFlow.

Certifications

- Infosys Springboard Data Science
- Udemy Data Analytics Course

Declaration

I hereby declare that all information furnished above is true to the best of my knowledge.

Date: May, 2026

Signature: _____

Curriculum Vitae

Full Name Ritvik Sharma
Roll No. 2200271540088
Branch Computer Science & Engineering (Data Science)
Batch 2022 – 2026
Date of Birth 25 July 2004
Email ritviksharmacse@gmail.com
Mobile +91-9560762861
LinkedIn linkedin.com/in/iamritvik



Educational Qualifications

Degree	Institution	Board/University	Year
B.Tech	AKGEC, Ghaziabad	AKTU	2026
Class XII	Gurukul - The School	CBSE	2022
Class X	Gurukul - The School	CBSE	2019

Internships & Experience

Summer Internship 2025 - IIT Indore *Jun 2025 – Jul 2025*

Worked on Image Encryption using AES and Arnold's Cat Map under Prof. Dr. Bhupesh Kr. Lad.

Google Developer Groups Member *Nov 2023 – Mar 2026*

Collaborating on hands-on ML projects and workshops to enhance AI skills.

Projects & Achievements

AES combined with Chaotic Maps *Jun 2025 – Jul 2025*

Designed a hybrid encryption system for digital images with pixel correlation analysis.

MMSI Hackathon (NEC & Mitsubishi Corp) *Achievement*

Acquired 1,20,000 cash reward for solving supply chain efficiency problems.

Certifications

- AWS Certified AI Practitioner (AIF C01)
- DUCAT Data Analytics & Artificial Intelligence
- Accenture Data Analytics & Visualization Virtual Experience

Declaration

I hereby declare that all information furnished above is true to the best of my knowledge.

Date: May, 2026

Signature: _____

Appendix E

Poster